LAYERZERO · SECURITY REVIEW

LayerZero Solana Console.

A security review of LayerZero's Console OFT Solana and Solana OApp contracts.

CLIENT	PROJECT	COMPONENTS	REVIEW WINDOW	FINAL REPORT
LayerZero	Solana Console	Console OFT · OApp Solana	April 10 – October 13, 2026	October 13, 2026

§ 00

Contents.

159 pages - overview, confidence ranking, scope, 113 findings, disclaimer, and about Guardian.

§ 01	Overview	p. 3
§ 02	Confidence Ranking	p. 4
§ 03	Scope & Classifications	p. 5
	Files in Scope	p. 6–14
§ 04	Findings	p. 15–23
	Console · Main Review · 15 findings	p. 24–38
	Console · Remediation Review 1 · 23 findings	p. 39–62
	Console · Remediation Review 2 · 4 findings	p. 63–66
	Console V2 · Main Review · 15 findings	p. 67–96
	Console V2 · Remediation Review 1 · 1 findings	p. 97–98
	OApp · Main Review 1 · 34 findings	p. 99–134
	OApp · Main Review 2 · 13 findings	p. 135–147
	OApp · Remediation Review 1 · 4 findings	p. 148–151
	OApp · Remediation Review 2 · 2 findings	p. 152–153
	OApp V2 · Main Review · 2 findings	p. 154–157
§ 05	Disclaimer	p. 158
§ 06	About Guardian	p. 159

§ 01

Overview.

Guardian reviewed LayerZero's Solana Console, covering the Console OFT Solana implementation and Solana OApp contracts across the main review and remediation phases. This consolidated report presents the scope, findings, and remediation outcomes for the integrated system.

CLIENT
LayerZero

PROJECT
Solana Console

CONSOLE REVIEW
May 11 – July 15, 2026

OAPP REVIEW
April 10 – June 23, 2026

OAPP + CONSOLE V2 REVIEW
September 29 – October 13, 2026

FINAL REPORT
October 13, 2026

REPOSITORY
https://github.com/LayerZero-Labs/monorepo-external

COMMIT
d88ec63859e261f9c33ed0d4c897909d72c3ad66

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5. No Critical or High findings were identified. The review surfaced two resolved Medium issues, Low-severity validation and documentation concerns, and predominantly Informational items. Findings are predominantly resolved or acknowledged following remediation review.

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	2	0	0	0	2
Low	19	0	3	0	16
Info	92	0	11	0	81

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. **Recommendation:** Code is highly secure at time of audit. Low risk of latent critical issues.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Only 1 Significant issue was uncovered per week. Design patterns are sound, and test coverage is strong. **Recommendation:** Suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity issues. Varied Low severity findings.
3 Moderate Confidence	Medium-severity and occasional High-severity issues found. Code is functional, but there are concerning areas such as weak modularity or risky patterns. No critical design flaws were identified, though some patterns could lead to edge-case issues. **Recommendation:** Address issues thoroughly and consider a targeted follow-up audit depending on code changes.	1-2 High/Critical findings per engagement week.
2 Low Confidence	Code shows frequent emergence of Critical/High vulnerabilities. Audit revealed recurring anti-patterns, weak test coverage, or unclear logic. These characteristics suggest a high likelihood of latent issues. **Recommendation:** Post-audit development and a second audit cycle are strongly advised.	2-4 High/Critical findings per engagement week, or additional unresolved High/Critical findings uncovered in remediation review.
1 Very Low Confidence	Code has systemic issues. Multiple High/Critical findings (>=5/week), poor security posture, and design flaws that introduce compounding risks. Safety cannot be assured. **Recommendation:** Halt deployment and seek a comprehensive re-audit after substantial refactoring.	>=5 High/Critical findings and overall systemic flaws.

§ 03

Scope & Classifications.

Guardian reviewed LayerZero's Solana Console across the `console_oft` and `console_transfer_hook` programs, together with the shared OFT implementation and instruction macros, as well as the Solana OApp source and macro generation, RBAC utilities, and shared Anchor trait framework code.

3.1 Severity Matrix

	IMP · HIGH	IMP · MEDIUM	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH	Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.
IMPACT · MEDIUM	Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.
IMPACT · LOW	Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.
LIKELIHOOD · HIGH	Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.
LIKELIHOOD · MEDIUM	Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.
LIKELIHOOD · LOW	Unlikely to ever occur in production.

§ 03

Files in Scope 1 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/oft-app/contracts/solana/oft/	
macros/src/instructions/mod.rs	18
macros/src/instructions/oft_type.rs	11
macros/src/instructions/quote_oft.rs	11
macros/src/instructions/quote_send.rs	11
macros/src/instructions/send.rs	11
macros/src/instructions/shared_decimals.rs	11
macros/src/instructions/token.rs	11
src/events.rs	36
src/lib.rs	5
src/oft_info.rs	68
src/types.rs	69
apps/project-types/console-oft-app/contracts/solana/	
Cargo.toml	57
apps/project-types/console-oft-app/contracts/solana/programs/console_oft/	
Cargo.toml	62
src/codec/compose_msg.rs	104
src/codec/mod.rs	2
src/codec/msg.rs	107
src/console_oft_info.rs	48
src/errors.rs	131
src/events.rs	151
src/helpers/credit_redirect.rs	27
src/helpers/mod.rs	64
src/helpers/pubkey_policy.rs	44
src/helpers/receive_accounts.rs	97
src/helpers/token2022.rs	140
src/helpers/transfer_hook_accounts.rs	100
src/helpers/transfer_hook_plan.rs	87

§ 03

Files in Scope 2 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/project-types/console-oft-app/contracts/solana/programs/console_oft/	
src/instructions/fee_config/fee_config_view.rs	154
src/instructions/fee_config/mod.rs	10
src/instructions/fee_config/set_default_fee_bps.rs	24
src/instructions/fee_config/set_fee_bps.rs	60
src/instructions/fee_handler/mod.rs	3
src/instructions/fee_handler/set_fee_deposit.rs	36
src/instructions/init_console_oft.rs	140
src/instructions/init_console_oft_info.rs	33
src/instructions/lz_receive.rs	423
src/instructions/lz_receive_types_info.rs	73
src/instructions/lz_receive_types_v3.rs	347
src/instructions/mod.rs	37
src/instructions/oft_store_view.rs	47
src/instructions/pause/mod.rs	7
src/instructions/pause/pause_config_view.rs	37
src/instructions/pause/set_default_paused.rs	37
src/instructions/pause/set_paused.rs	76
src/instructions/quote_oft.rs	97
src/instructions/quote_send.rs	80
src/instructions/rate_limiter/checkpoint_rate_limit.rs	69
src/instructions/rate_limiter/get_rate_limit_usages.rs	45
src/instructions/rate_limiter/is_rate_limit_address_exempt.rs	25
src/instructions/rate_limiter/mod.rs	17
src/instructions/rate_limiter/rate_limits.rs	30
src/instructions/rate_limiter/set_rate_limit_address_exemption.rs	69
src/instructions/rate_limiter/set_rate_limit_config.rs	59
src/instructions/rate_limiter/set_rate_limit_global_config.rs	40
src/instructions/rate_limiter/set_rate_limit_state.rs	69
src/instructions/send.rs	485
src/instructions/set_credit_redirect_config.rs	48

§ 03

Files in Scope 3 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/project-types/console-oft-app/contracts/solana/programs/console_oft/	
src/instructions/set_pubkey_policy.rs	37
src/instructions/set_token_manager.rs	30
src/instructions/sync_acl_gating_program.rs	45
src/instructions/sync_transfer_hook_program.rs	46
src/lib.rs	263
src/seeds.rs	5
src/state/mod.rs	130
src/state/oft.rs	93
src/state/rate_limit.rs	703
apps/project-types/oapp-app/contracts/solana/	
Cargo.toml	30
macros/Cargo.toml	23
macros/src/composer.rs	41
macros/src/instructions/composer/lz_compose.rs	17
macros/src/instructions/composer/lz_compose_types_info.rs	29
macros/src/instructions/composer/lz_compose_types_v3.rs	18
macros/src/instructions/composer/mod.rs	23
macros/src/instructions/mod.rs	4
macros/src/instructions/oapp/accept_default_admin_transfer.rs	50
macros/src/instructions/oapp/burn.rs	69
macros/src/instructions/oapp/clear.rs	70
macros/src/instructions/oapp/get_enforced_options.rs	57
macros/src/instructions/oapp/get_peer.rs	55
macros/src/instructions/oapp/init_enforced_options.rs	87
macros/src/instructions/oapp/is_compose_msg_sender.rs	44
macros/src/instructions/oapp/lz_receive.rs	33
macros/src/instructions/oapp/lz_receive_types_info.rs	26
macros/src/instructions/oapp/lz_receive_types_v3.rs	17
macros/src/instructions/oapp/mod.rs	49

§ 03

Files in Scope 4 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/project-types/oapp-app/contracts/solana/	
macros/src/instructions/oapp/next_nonce.rs	53
macros/src/instructions/oapp/nilify.rs	69
macros/src/instructions/oapp/set_enforced_options.rs	101
macros/src/instructions/oapp/set_peer.rs	74
macros/src/instructions/oapp/skip.rs	68
macros/src/lib.rs	100
macros/src/oapp.rs	80
macros/src/state_types.rs	79
src/common.rs	74
src/endpoint_cpi.rs	549
src/errors.rs	17
src/events.rs	26
src/lib.rs	96
src/lz_compose_types_v3.rs	72
src/lz_plan.rs	71
src/lz_receive_types_v3.rs	120
src/oapp_info.rs	278
src/options.rs	137
src/types.rs	158
src/utils.rs	14
apps/token-app/contracts/solana/pubkey-policy-hook/	
Cargo.toml	21
apps/token-app/contracts/solana/pubkey-policy-hook/programs/pubkey_policy_hook/	
Cargo.toml	36
src/errors.rs	8
src/events.rs	8
src/instructions/init_transfer_hook.rs	97
src/instructions/mod.rs	5

§ 03

Files in Scope 5 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/token-app/contracts/solana/pubkey-policy-hook/programs/pubkey_policy_hook/	
src/instructions/transfer_hook.rs	85
src/lib.rs	33
src/state.rs	51
apps/token-app/contracts/solana/token-manager/	
Cargo.toml	48
interface/Cargo.toml	21
interface/src/acl/manifest.rs	283
interface/src/acl/mod.rs	36
interface/src/acl/thaw.rs	171
interface/src/burn.rs	47
interface/src/lib.rs	16
interface/src/seeds.rs	30
interface/src/state.rs	58
interface/src/version.rs	21
apps/token-app/contracts/solana/token-manager/programs/token_manager/	
Cargo.toml	41
src/authority.rs	46
src/errors.rs	28
src/events.rs	80
src/instructions/acl/freeze_token_account.rs	65
src/instructions/acl/gate.rs	88
src/instructions/acl/helpers.rs	88
src/instructions/acl/init_acl.rs	86
src/instructions/acl/manage_acl.rs	104
src/instructions/acl/mod.rs	12
src/instructions/acl/thaw_token_account.rs	65
src/instructions/get_version.rs	8
src/instructions/mod.rs	13

§ 03

Files in Scope 6 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
apps/token-app/contracts/solana/token-manager/programs/token_manager/	
src/instructions/native_authority/mod.rs	3
src/instructions/native_authority/set_native_authority.rs	111
src/instructions/pause/set_paused.rs	142
src/instructions/permanent_delegate/mod.rs	5
src/instructions/permanent_delegate/pd_burn.rs	114
src/instructions/permanent_delegate/pd_recover.rs	146
src/instructions/permissioned_burn/burn.rs	123
src/instructions/permissioned_burn/mod.rs	3
src/instructions/root/init_token_manager.rs	113
src/instructions/root/mod.rs	5
src/instructions/root/rbac_default_admin.rs	69
src/lib.rs	133
src/state.rs	3
src/types.rs	36
contracts/common/framework/solana/anchor-trait/	
Cargo.toml	39
src/assembly.rs	105
src/generation.rs	205
src/lib.rs	134
src/overrides.rs	271
contracts/common/utils/solana/heap-allocator/	
Cargo.toml	6
lib/Cargo.toml	12
lib/src/lib.rs	95
contracts/common/utils/solana/program-id/	
Cargo.toml	6
lib/Cargo.toml	16

§ 03

Files in Scope 7 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
contracts/common/utils/solana/program-id/	
lib/src/lib.rs	57
contracts/common/utils/solana/pubkey-policy/	
Cargo.toml	24
interface/Cargo.toml	17
interface/src/error.rs	31
interface/src/lib.rs	19
interface/src/pubkey_policy.rs	257
interface/src/seeds.rs	20
interface/src/state.rs	69
interface/src/version.rs	21
contracts/common/utils/solana/pubkey-policy/programs/pubkey_policy/	
Cargo.toml	32
src/errors.rs	11
src/events.rs	27
src/instructions/get_version.rs	8
src/instructions/init_pubkey_policy.rs	56
src/instructions/membership.rs	23
src/instructions/mod.rs	12
src/instructions/set_blacklisted.rs	56
src/instructions/set_mode.rs	28
src/instructions/set_whitelisted.rs	56
src/lib.rs	58
src/types.rs	29
contracts/common/utils/solana/rbac/	
Cargo.toml	37
macros/Cargo.toml	16
macros/src/init_default_admin.rs	73

§ 03

Files in Scope 8 / 8.

https://github.com/LayerZero-Labs/monorepo-external

FILE	SOURCE LINES
contracts/common/utils/solana/rbac/	
macros/src/instructions/accept_default_admin_transfer.rs	115
macros/src/instructions/begin_default_admin_transfer.rs	60
macros/src/instructions/get_default_admin.rs	47
macros/src/instructions/grant_role.rs	110
macros/src/instructions/mod.rs	29
macros/src/instructions/renounce_role.rs	85
macros/src/instructions/revoke_role.rs	103
macros/src/lib.rs	170
macros/src/only_role.rs	51
macros/src/rbac.rs	68
macros/src/role_type.rs	65
macros/src/setup_default_admin.rs	113
macros/src/state_types.rs	98
macros/src/utils.rs	64
src/errors.rs	45
src/events.rs	41
src/lib.rs	84
src/traits.rs	94
src/types.rs	35

§ 03

Historical Files in Scope 1 / 1.

https://github.com/LayerZero-Labs/monorepo-external

These 24 files were covered in historical reviews and are absent from the referenced commit. Original paths and source line counts are retained.

FILE	SOURCE LINES
apps/project-types/console-oft-app/contracts/solana/programs/console_oft/	
src/instructions/lz_receive_types_v2.rs	241
src/instructions/set_alts.rs	29
apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/	
Cargo.toml	44
build.rs	13
src/errors.rs	30
src/events.rs	37
src/instructions/allowlist/allowlist_view.rs	26
src/instructions/allowlist/mod.rs	8
src/instructions/allowlist/set_allowlist_mode.rs	21
src/instructions/allowlist/set_blacklisted.rs	45
src/instructions/allowlist/set_whitelisted.rs	40
src/instructions/init_transfer_hook.rs	138
src/instructions/init_transfer_hook_info.rs	23
src/instructions/mod.rs	14
src/instructions/pause/mod.rs	4
src/instructions/pause/pause.rs	17
src/instructions/pause/unpause.rs	17
src/instructions/set_bypass.rs	47
src/instructions/transfer_hook.rs	109
src/instructions/transfer_hook_view.rs	17
src/lib.rs	84
src/state.rs	73
src/transfer_hook_info.rs	47
src/types.rs	72

§ 04

Findings.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
CONSOLE · MAIN REVIEW					
I-01	Allowlist Mode ABI Uses Raw u8.	Compatibility	INFO	RESOLVED	p. 24
I-02	Parity doc misdescribes rate_limits return type.	Documentation	INFO	RESOLVED	p. 25
I-03	FeeDepositSet doc names nonexistent EVM event.	Documentation	INFO	RESOLVED	p. 26
I-04	Parity doc references stale scale_decimals field.	Documentation	INFO	RESOLVED	p. 27
I-05	OftInfo.oft_type raw u8 bypasses enum guard.	Best Practices	INFO	RESOLVED	p. 28
I-06	Undocumented RoleType requirement in #[oft].	Documentation	INFO	RESOLVED	p. 29
I-07	Rate limiter doc names nonexistent EVM file.	Documentation	INFO	RESOLVED	p. 30
I-08	getRateLimitUsages doc misnames EVM params.	Documentation	INFO	RESOLVED	p. 31
I-09	Unqualified Pubkey Makes Expansion Import-Sensitive.	Compatibility	INFO	ACKNOWLEDGED	p. 32
I-10	Inconsistent OFT type docs.	Documentation	INFO	RESOLVED	p. 33
I-11	Misleading oft_app comment.	Documentation	INFO	RESOLVED	p. 34
I-12	fee_amount_ld Uses Narrower Integer Range.	Informational	INFO	RESOLVED	p. 35
I-13	Document write handler authorization.	Documentation	INFO	RESOLVED	p. 36
I-14	OFT TLV Metadata Misreports Account Layouts.	Informational	INFO	RESOLVED	p. 37
I-15	Blacklist checks miss token account owners.	Logical Error	INFO	ACKNOWLEDGED	p. 38

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
CONSOLE · REMEDIATION REVIEW 1					
L-01	Quote Can Fail When Global Limits Are Disabled.	DoS	LOW	RESOLVED	p. 39
L-02	Rate-Limit No-Op Cases Still Require Per-EID PDA.	Unexpected Behavior	LOW	RESOLVED	p. 40
L-03	BurnMint Whitelist- Mode Escrow Send DoS.	Compatibility	LOW	RESOLVED	p. 41
L-04	Missing Mint Authority Check During Initialize.	Validation	LOW	ACKNOWLEDGED	p. 42
L-05	Enforced options msg_type mismatch.	Configuration	LOW	RESOLVED	p. 43
I-01	Transfer Hook Index- 3 Authority Doc Mislabel.	Documentation	INFO	RESOLVED	p. 44
I-02	Document Solana Compose Codec Layout.	Documentation	INFO	RESOLVED	p. 45
I-03	Shared Program Upgrades Affect All OFT Instances.	Trust Assumptions	INFO	RESOLVED	p. 46
I-04	Dust Can Misalign Hook Accounts.	Documentation	INFO	RESOLVED	p. 47
I-05	RoleType is always required.	Informational	INFO	RESOLVED	p. 48
I-06	Receive-Types Token Mint Comment Mismatch.	Informational	INFO	RESOLVED	p. 49
I-07	Mint Bypass Comment Names Wrong Modifier.	Informational	INFO	RESOLVED	p. 50
I-08	Fees Should Account For Executor Funded ATA Creation.	Documentation	INFO	RESOLVED	p. 51
I-09	Document Escrow Entries In Whitelist Mode.	Documentation	INFO	RESOLVED	p. 52

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
CONSOLE · REMEDIATION REVIEW 1					
I-10	QuoteOFT Max Understates Sendable Amount.	Unexpected Behavior	INFO	RESOLVED	p. 53
I-11	recoverFunds Parity Comment Uses Wrong Signature.	Documentation	INFO	RESOLVED	p. 54
I-12	Fund-Recovery Authority Rotation Undocumented.	Documentation	INFO	RESOLVED	p. 55
I-13	Rate Exemption Seed Name Typo.	Documentation	INFO	RESOLVED	p. 56
I-14	Per-ID PDA Seed Width Docs.	Documentation	INFO	RESOLVED	p. 57
I-15	Rate Limiter Docs Cite Wrong File.	Documentation	INFO	RESOLVED	p. 58
I-16	Pause Docs Cite Wrong EVM Contract.	Documentation	INFO	RESOLVED	p. 59
I-17	Misleading fee inverse edge cases.	Documentation	INFO	RESOLVED	p. 60
I-18	Arbitrary compose plans expose Executor tokens.	Validation	INFO	ACKNOWLEDGED	p. 61
CONSOLE · REMEDIATION REVIEW 2					
L-01	Missing RateLimit PDA mismatch.	Logical Error	LOW	RESOLVED	p. 63
I-01	Informational Note Regarding Rate Limit Test.	Informational	INFO	RESOLVED	p. 64
I-02	Rate-limit PDA requirement is underdocumented.	Documentation	INFO	RESOLVED	p. 65
I-03	Misleading Globally-Disabled Usage Doc.	Documentation	INFO	RESOLVED	p. 66

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
CONSOLE V2 · MAIN REVIEW					
M-01	init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one	Compatibility	MEDIUM	RESOLVED	p. 67
L-01	A recipient can close its empty frozen ATA to receive a credit the docs say is redirected	Validation	LOW	RESOLVED	p. 72
L-03	Confidential balances cannot be recovered or burned by pd_recover and pd_burn	Logical Error	LOW	RESOLVED	p. 74
L-04	Anyone can freeze the empty BurnMint escrow once the policy blocks oft_store, halting the OFT	DoS	LOW	RESOLVED	p. 76
L-05	lz_receive_types_info reads a synced default ACL gate as "no config", so every receive fails to plan while a policy is selected	Logical Error	LOW	RESOLVED	p. 78
I-01	Redirect escrow requirements omit memo-required and confidential-only accounts	Documentation	INFO	RESOLVED	p. 80
I-02	A zero-address recipient is reported as fully credited on Solana but as 0 on EVM	Documentation	INFO	RESOLVED	p. 82
I-03	Conditional native-authority signer-meta requirements are not documented	Documentation	INFO	RESOLVED	p. 84
I-05	The parity doc's advice to hold an empty account by blacklisting has no effect in Open mode	Documentation	INFO	RESOLVED	p. 85
I-06	The successor-OFT guidance tells operators to move LockUnlock collateral, but Console OFT has no instruction or procedure for it	Documentation	INFO	RESOLVED	p. 87

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
CONSOLE V2 · MAIN REVIEW					
I-07	pd_recover credits the destination net of Token-2022 transfer fees while FundsRecovered reports the gross amount	Documentation	INFO	RESOLVED	p. 89
I-08	The README names the wrong error for init_acl rejecting another mint's policy	Documentation	INFO	RESOLVED	p. 90
I-09	The README's pause-authority setup is one pd_recover cannot resume	Documentation	INFO	RESOLVED	p. 91
I-10	The README says a zero-fee send can omit the fee deposit, but decimal dust still requires it	Documentation	INFO	RESOLVED	p. 93
I-11	The successor TokenManager migration has the old root hand over the authority step 1 already moved to a signing key	Documentation	INFO	RESOLVED	p. 95
CONSOLE V2 · REMEDIATION REVIEW 1					
I-01	set_acl_permissions reverts on unchanged flags "as on EVM", but EVM's two-flag setter stores and emits	Documentation	INFO	RESOLVED	p. 97
OAPP · MAIN REVIEW 1					
M-01	OApp macro rejects module inner attributes.	Logical Error	MEDIUM	RESOLVED	p. 99
L-01	Payer-funder mismatch in enforced options.	Logical Error	LOW	RESOLVED	p. 101
L-02	Admin transfer reverts if OApp not registered.	Validation	LOW	RESOLVED	p. 102
L-04	Missing #[repr(u8)] enable role seed collision.	Validation	LOW	RESOLVED	p. 103

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
OAPP · MAIN REVIEW 1					
L-05	only_role Does Not Enforce Signer.	Validation	LOW	RESOLVED	p. 104
L-06	Fully-qualified attribute paths not matched.	Validation	LOW	RESOLVED	p. 105
L-07	Override Handler Cannot Access Nested Module Helpers.	Logical Error	LOW	RESOLVED	p. 106
L-08	Peer/options accounts cannot be closed to reclaim rent.	Best Practices	LOW	ACKNOWLEDGED	p. 108
I-01	EnforcedOptions space vs field order mismatch.	Informational	INFO	RESOLVED	p. 109
I-02	Override can desync OAppInfo and SDK discovery.	Validation	INFO	RESOLVED	p. 110
I-03	Misleading set_delegate mention in macro doc.	Informational	INFO	RESOLVED	p. 111
I-04	lz_receive_types_info accounts undocumented.	Best Practices	INFO	RESOLVED	p. 112
I-05	Unused error variant OAppError::InvalidPeer.	Informational	INFO	RESOLVED	p. 113
I-06	RBAC role-admin mapping is fixed at compile time.	Informational	INFO	ACKNOWLEDGED	p. 114
I-07	Marker attribute macro bodies are unreachable.	Informational	INFO	RESOLVED	p. 115
I-08	Duplicate use super::* in generated module.	Informational	INFO	ACKNOWLEDGED	p. 116
I-09	endpoint_cpi Helpers Can Panic on Short Slices.	Validation	INFO	RESOLVED	p. 117
I-10	Events omit the state instance pubkey.	Events	INFO	RESOLVED	p. 118

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
OAPP · MAIN REVIEW 1					
I-11	The default_admin_role should not be dynamic.	Warning	INFO	RESOLVED	p. 119
I-12	RoleRevoked.sender doc copy-pasted from grant.	Informational	INFO	RESOLVED	p. 120
I-13	Clarify endpoint program check in CPI wrappers.	Documentation	INFO	RESOLVED	p. 121
I-14	Misleading Code Comment.	Documentation	INFO	RESOLVED	p. 122
I-15	Endpoint CPI wrappers rely on account ordering.	Suggestion	INFO	RESOLVED	p. 123
I-16	OApp initialize() is frontrun-hijackable.	Frontrunning	INFO	ACKNOWLEDGED	p. 124
I-17	RBAC docs reference non-existent #[grant_role].	Documentation	INFO	RESOLVED	p. 125
I-18	deserialize_alt Can Panic Via unwrap.	Informational	INFO	RESOLVED	p. 126
I-19	Hardcoded crate path in declare_instructions.	Compatibility	INFO	RESOLVED	p. 127
I-20	Arbitrary marker accepted instead of (ctx).	Compatibility	INFO	RESOLVED	p. 128
I-21	Instruction aliases emit duplicate entrypoints.	Documentation	INFO	RESOLVED	p. 129
I-22	Docs understate default impl remains available.	Documentation	INFO	RESOLVED	p. 130
I-23	Default accounts generics are not supported.	Compatibility	INFO	RESOLVED	p. 131
I-24	Field-Injection Macros Always Append New Fields.	Documentation	INFO	RESOLVED	p. 132
I-25	OAppInfo can be silently uninitialized.	Compatibility	INFO	RESOLVED	p. 133

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
OAPP · MAIN REVIEW 1					
I-26	Generated Code Is Susceptible To Name Shadowing.	Compatibility	INFO	RESOLVED	p. 134
OAPP · MAIN REVIEW 2					
L-01	OAppInfo Can Reference Missing TLV Entry.	Validation	LOW	ACKNOWLEDGED	p. 135
I-01	Unconstrained sender in setup_default_admin!.	Events	INFO	RESOLVED	p. 136
I-02	clear wrapper docs imply OApp-only signer.	Validation	INFO	RESOLVED	p. 137
I-03	Stale OAppInfo Documentation After TLV Migration.	Documentation	INFO	RESOLVED	p. 138
I-04	anchor-trait README stale on wrapper_defs.	Documentation	INFO	RESOLVED	p. 139
I-05	Consider Size Check In update_tlv_entry.	Validation	INFO	RESOLVED	p. 140
I-06	Incorrect composer docs link.	Documentation	INFO	RESOLVED	p. 141
I-07	Private repo link in source comment.	Documentation	INFO	RESOLVED	p. 142
I-08	Incomplete package files list.	Configuration	INFO	RESOLVED	p. 143
I-09	register_oapp Prerequisite Documentation Gap.	Documentation	INFO	RESOLVED	p. 144
I-10	OAppInfo init accepts nonzero TLV tail.	Validation	INFO	RESOLVED	p. 145
I-11	Program attr filtering should be documented.	Documentation	INFO	RESOLVED	p. 146
I-12	RoleType derive can use nonzero default.	Validation	INFO	RESOLVED	p. 147

§ 04

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
OAPP · REMEDIATION REVIEW 1					
I-01	Composer codegen doc points to the wrong file.	Documentation	INFO	ACKNOWLEDGED	p. 148
I-02	Typo in CallerNotPendingAdmin doc comment.	Documentation	INFO	ACKNOWLEDGED	p. 149
I-03	Broken intra-doc link to CodegenContext.	Best Practices	INFO	ACKNOWLEDGED	p. 150
I-04	Misleading clear documentation.	Documentation	INFO	ACKNOWLEDGED	p. 151
OAPP · REMEDIATION REVIEW 2					
I-01	Stale OAppBase doc snippet in oapp-info.	Documentation	INFO	ACKNOWLEDGED	p. 152
I-02	OAppInfo v1 Can Allow Misleading Discovery.	Documentation	INFO	RESOLVED	p. 153
OAPP V2 · MAIN REVIEW					
L-02	The compose docs say lz_compose has no sender to check, but anyone can queue a compose message	Documentation	LOW	RESOLVED	p. 154
I-04	The README says a permanently blocked receive can be skipped, but skip cannot reach a verified payload	Documentation	INFO	RESOLVED	p. 156

INFO I-01 RESOLVED

Allowlist Mode ABI Uses Raw u8.

CATEGORY Compatibility

LOCATION apps/oft-app/contracts/solana/transfer-hook/macros/src/instructions/allowlist/set_allowlist_mode.rs#L7

REVIEW Console · Main Review

DESCRIPTION

The transfer-hook interface currently defines `set_allowlist_mode` with a raw `u8` parameter, so the instruction ABI can carry any value between 0-255. However, the intended domain for allowlist mode is only three enum values: `Open`, `Blacklist`, and `Whitelist`. The framework includes `TryFrom<u8>` for `AllowlistMode`, which correctly constrains `u8` to those three values, but because the ABI type is still u8, downstream projects must remember to call `AllowlistMode::try_from` themselves in each relevant handler (for example in initialize). If validation is omitted or implemented incorrectly, invalid mode values can reach business logic and cause unexpected behaviors that differs from the intended allowlist policy.

RECOMMENDATION

Either explicitly document as a requirement that all implementations should validate mode inputs using `AllowlistMode::try_from`, or consider changing the instruction parameter type from `u8` to `AllowlistMode` so invalid values are rejected at the interface boundary by construction.

RESOLUTION

Status: Resolved.

INFO I-02 RESOLVED

Parity doc misdescribes rate_limits return type.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/macros/src/instructions/rate_limiter/rate_limits.rs#L13

REVIEW Console · Main Review

DESCRIPTION

The parity doc (`solana-evm-parity-analysis.md` §4.1, §4.8) documents `rate_limits` as returning bare `RateLimit` , matching the EVM `IRateLimiter.rateLimits(uint256) → RateLimit` signature. The actual spec in `oft-extended/macros/src/instructions/rate_limiter/rate_limits.rs` returns `Option<RateLimit>` : `return_type: quote! (Option<::oft_extended::types::RateLimit>),` `Option<RateLimit>` is the correct Solana shape - per-ID rate-limit state lives in a PDA that may or may not exist (`None` = no PDA initialized for that EID, `Some(v)` = configured). Off-chain SDKs that bind from the EVM-parity doc rather than the IDL misdecode the response: the `Option` tag byte (0x00/0x01) is read as the first field of `RateLimit` (`config.override_default_config: bool`), producing a one-byte-misaligned decode for every `rate_limits` query.

In addition, on Solana the `RateLimit` struct fields are inverted - config is first, state is second. This is opposite on EVM. `pub struct RateLimit { pub config: RateLimitConfig, pub state: RateLimitState, }` The docs also present the `RateLimit` struct as if it's a flat struct, but in reality it's wrapping `RateLimitConfig` and `RateLimitState`

RECOMMENDATION

Update the doc (no code change). In §4.1 row 2, change the Solana `Return` column from `RateLimit` to `Option<RateLimit>` . In §4.8, prepend a paragraph using the same template as §1.5/§3.3 (" `Some(v)` = per-ID PDA exists; `None` = no PDA, fall back to default"). Also update the docs to reflect the fact that `RateLimit` is not flat. Consider if reversing the order is desired.

RESOLUTION

Status: Resolved.

INFO I-03 RESOLVED

FeeDepositSet doc names nonexistent EVM event.

CATEGORY Documentation LOCATION apps/oft-app/contracts/solana/oft-extended/src/events.rs#L100

REVIEW Console · Main Review

DESCRIPTION

The event doc in `oft-extended/src/events.rs` cites an EVM event named `FeeDepositUpdated`, which does not exist in the EVM interfaces: `/// Emitted when the fee deposit is updated. /// EVM: event FeeDepositUpdated(address indexed feeDeposit) #[event] pub struct FeeDepositSet { pub oft_store: Pubkey, pub fee_deposit: Pubkey, }` The actual EVM event declared in https://github.com/LayerZero-Labs/monorepo-external/blob/main/contracts/common/utils/evm/non-upgradeable/contracts/interfaces/IFeeHandler.sol#L20 is `event FeeDepositSet(address indexed feeDeposit);`. The Solana event name (`FeeDepositSet`) matches the EVM event name exactly; only the doc comment misnames the EVM equivalent.

RECOMMENDATION

Update the doc comment to reference the actual EVM event: `-/// EVM: event FeeDepositUpdated(address indexed feeDeposit) +/// EVM: event FeeDepositSet(address indexed feeDeposit)`

RESOLUTION

Status: Resolved.

INFO I-04 RESOLVED

Parity doc references stale scale_decimals field.

CATEGORY Documentation LOCATION apps/oft-app/contracts/solana/oft-extended/src/types.rs#L58-L66

REVIEW Console · Main Review

DESCRIPTION

The parity doc (`solana-evm-parity-analysis.md` §4.2, §5.2) documents `RateLimitGlobalConfig` as `{ use_global_state, is_globally_disabled, scale_decimals: u8 }` and explicitly lists `scale_decimals` as a "Solana-only addition" that is "runtime-configurable" (vs. the EVM `SCALE_DECIMALS` immutable). The actual struct in `oft-extended/src/types.rs` ships only the two bools: `pub struct RateLimitGlobalConfig { pub use_global_state: bool, pub is_globally_disabled: bool, }` `SetRateLimitGlobalConfigParams` matches the same two-field shape, and no separate setter for `scale_decimals` exists among the 21 oft-extended instructions. Off-chain SDKs and integrators that bind from the EVM-parity doc misdecode `RateLimitGlobalConfig` (extra byte read into a non-existent field).

RECOMMENDATION

Consider deleting the `scale_decimals` references from the parity doc so it matches the code.

RESOLUTION

Status: Resolved.

INFO I-05 RESOLVED

OftInfo.oft_type raw u8 bypasses enum guard.

CATEGORY Best Practices LOCATION apps/oft-app/contracts/solana/oft/src/oft_info.rs#L44

REVIEW Console · Main Review

DESCRIPTION

`OftType` (`oft/src/oft_info.rs:19`) is a two-variant enum with `#[borsh(use_discriminant = true)]`, which would reject any byte outside `{0, 1}` during deserialization. But `OftInfo` (`oft_info.rs`) stores the custody model as a raw `u8`, not as `OftType`: `pub struct OftInfo { pub version: u8, pub oft_type: u8, // raw byte, not OftType pub instructions: OftInstructionVersions, pub project_discriminator: [u8; 8], }` All fields are `pub`. The safe constructor `OftInfo::new(oft_type: OftType, …)` takes the enum and casts it (`oft_type as u8`), but a caller can also build the struct directly (`OftInfo { oft_type: 99, … }`) without a compile-time check or discriminant guard on read. Downstream consumers pull a raw `u8` and must validate themselves.

RECOMMENDATION

Type the field as the enum so the Borsh guard applies on both write and read paths: `pub struct OftInfo { pub version: u8, pub oft_type: OftType, pub instructions: OftInstructionVersions, pub project_discriminator: [u8; 8], }`

RESOLUTION

Status: Resolved.

INFO I-06 RESOLVED

Undocumented RoleType requirement in #[oft].

CATEGORY Documentation LOCATION apps/oft-app/contracts/solana/oft/macros/src/oft.rs#L46

REVIEW Console · Main Review

DESCRIPTION

The `#[oft]` macro injects `#[::oapp::oapp(state = #oft_type, role_type = crate::RoleType)]` (`oft/macros/src/oft.rs:46`). The `role_type` path is hardcoded - consumers must declare a `RoleType` enum at crate root with that exact name. `#[oft_extended]` inherits this via its nested `#[oft]` injection. This implicit requirement isn't mentioned in any documentation. Consumers only discover it via compile errors.

RECOMMENDATION

Add documentation stating that consumers must declare a `RoleType` enum at crate root satisfying the `rbac` derive contract.

RESOLUTION

Status: Resolved.

INFO I-07 RESOLVED

Rate limiter doc names nonexistent EVM file.

CATEGORY Documentation LOCATION apps/oft-app/contracts/solana/oft-extended/src/events.rs#L56

REVIEW Console · Main Review

DESCRIPTION

The events section in `oft-extended/src/events.rs` claims EVM alignment with a file that does not exist: `// ================================ Rate Limiter Events ================================ // EVM alignment: IRateLimiterCore.sol` No `IRateLimiterCore.sol` exists anywhere. The actual EVM interface declaring all four rate-limiter events in this section (`RateLimitConfigUpdated`, `RateLimitGlobalConfigUpdated`, `RateLimitStateUpdated`, `RateLimitAddressExemptionUpdated`) is `IRateLimiter.sol`.

RECOMMENDATION

Update the comment to reference the actual EVM interface file.

RESOLUTION

Status: Resolved.

INFO I-08 RESOLVED

getRateLimitUsages doc misnames EVM params.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft-extended/macros/src/instructions/rate_limiter/get_rate_limit_usages.rs#L3-L5

REVIEW Console · Main Review

DESCRIPTION

The macro doc in `get_rate_limit_usages.rs` cites the EVM return-parameter names incorrectly: `//! EVM: IRateLimiter.getRateLimitUsages(uint256 _id) → //! (uint256 outboundUsage, uint256 outboundAvailable, uint256 inboundUsage, uint256 //! inboundAvailable)` The actual EVM signature declared in `IRateLimiter.sol` is: `function getRateLimitUsages( uint256 _id ) external view returns ( uint256 outboundUsage, uint256 outboundAvailableAmount, uint256 inboundUsage, uint256 inboundAvailableAmount );`

RECOMMENDATION

Update the doc comment to reference the actual EVM return parameter names.

RESOLUTION

Status: Resolved.

INFO I-09

ACKNOWLEDGED

Unqualified Pubkey Makes Expansion Import-Sensitive.

CATEGORY Compatibility

LOCATION apps/oft-app/contracts/solana/oft/macros/src/instructions/token.rs#L13 and related helpers

REVIEW Console · Main Review

DESCRIPTION

Several OFT / OFT-extended instruction specs emit bare `Pubkey` types in generated signatures instead of fully qualified paths. As a result, macro expansion depends on the downstream module already importing `Pubkey` into scope, so otherwise correct implementations may fail to compile for reasons unrelated to their business logic. While this is a compile-time developer footgun rather than a runtime issue, it reduces the robustness and portability of these framework macros for third-party consumers.

RECOMMENDATION

Emit fully qualified paths for generated public types, such as `::anchor_lang::prelude::Pubkey`, so macro expansion is independent of downstream imports.

RESOLUTION

Status: Acknowledged.

The current design was retained because using the fully qualified type breaks Anchor IDL generation; consumers still need to import `Pubkey`.

INFO I-10 RESOLVED

Inconsistent OFT type docs.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft/src/types.rs#L40 apps/oft-app/contracts/solana/oft/src/types.rs#L104

REVIEW Console · Main Review

DESCRIPTION

The `///` documentation for `QuoteOFTResult` states that it provides the fee breakdown, limits, and receipt data, but both the struct field order and the documented `IOFT.quoteOFT()` return tuple order are `OFTLimit`, `OFTFeeDetail[]`, and `OFTReceipt`. The `///` documentation for `QuoteOFTParams` states that it provides the fee breakdown and settings data for an OFT, but `QuoteOFTParams` is the input parameter object used by `quoteOFT()`. The fee breakdown and receipt-related data are returned through `QuoteOFTResult`, not provided by the params type.

RECOMMENDATION

Update the `QuoteOFTResult` documentation to list the returned data in the same order as the struct and `IOFT.quoteOFT()` return tuple: limits, fee breakdown, and receipt data. Make the documentation describe the struct as the parameters used to quote an OFT transfer, rather than as data that provides fee breakdown or settings information.

RESOLUTION

Status: Resolved.

INFO I-11 RESOLVED

Misleading oft_app comment.

CATEGORY Documentation

LOCATION apps/oft-app/contracts/solana/oft/src/oft_info.rs#L34-L35 apps/oft-app/contracts/solana/oft/src/oft_info.rs#L48-L49

REVIEW Console · Main Review

DESCRIPTION

There are 2 comments about `OftInfo` which suggest it's possible for an SDK to jump directly to a `TLV` entry without scanning the buffer thanks to the `OAppBase.app_discriminator` and `OftInfo.project_discriminator`. This is misleading, since the discriminators are used for type discovery, but they cannot serve the goal of direct jump, as there is no offset encoded in them.

RECOMMENDATION

Clear up the comments.

RESOLUTION

Status: Resolved.

INFO I-12 RESOLVED

fee_amount_ld Uses Narrower Integer Range.

CATEGORY Informational LOCATION apps/oft-app/contracts/solana/oft/src/types.rs#L34

REVIEW Console · Main Review

DESCRIPTION

`OFTFeeDetail.fee_amount_ld` is defined as `i64`, while the broader OFT amount model uses `u64`. As a result, `fee_amount_ld` supports a narrower value range than other token amount fields in the interface. Although fee values are unlikely to approach these limits in typical Solana deployments, the reduced range may still be relevant for integrations or tooling that assume fee metadata can represent the full token amount domain.

RECOMMENDATION

Consider using a signed type that fully covers the u64 amount domain, such as `i128`, or explicitly documenting that `fee_amount_ld` is intentionally narrower as part of the interface design.

RESOLUTION

Status: Resolved.

INFO I-13 RESOLVED

Document write handler authorization.

CATEGORY Documentation LOCATION oft, oft_extended REVIEW Console · Main Review

DESCRIPTION

The `oft_extended` documentation states that required instructions such as `set_rate_limit_state()` have no default implementation and must be implemented by concrete OFT programs, but it does not clearly state that write handlers must also implement authorization matching the EVM RBAC surface. This is easy to miss because the macro only enforces that a handler exists; it does not enforce that the handler includes `#[rbac::only_role(...)]` or another access-control check.

RECOMMENDATION

Document that required OFT Extended write handlers must add authorization explicitly, because the macro does not provide default RBAC.

RESOLUTION

Status: Resolved.

INFO I-14 RESOLVED

OFT TLV Metadata Misreports Account Layouts.

CATEGORY Informational LOCATION oft_info.rs REVIEW Console · Main Review

DESCRIPTION

OftInfo tells SDKs that all base OFT instructions use standard account layout version 1, but the OFT framework does not define or enforce any standard account layout for those instructions All five OFT instructions are required overrides. Each concrete program chooses its own Context<T> account layout, yet the shared metadata still advertises them as standard layouts This can cause generic SDKs or executors that trust OAppInfo / OftInfo to build the wrong account list, making core OFT actions like send, quote_send, and quote_oft fail for users OftInfo hardcodes every base OFT instruction version to 1 in OftInstructionVersions OftInfo::new() then publishes those versions during initialization But the OFT macro does not provide predefined account templates for these instructions, every OFT instruction has `default_impl: None`

So the concrete program must implement the handler, and anchor-trait dispatches using the handler declared Context<T>

Example `#[oft_instruction] pub fn send( ctx: &mut Context<MyCustomSendAccounts>, params: &SendParams, ) → Result<(MessagingReceipt, OFTReceipt)> { }` The metadata says send = 1, but the real account layout is MyCustomSendAccounts In console_oft for example, the mismatch is real send requires pause, fee, rate-limit, exemption, token source, escrow, fee deposit, mint, token program, and Endpoint accounts Those are concrete program specific layouts, not framework generated OFT standard layouts A metadata driven SDK can read `OftInfo.instructions.send = 1` and take the standard discovery path, even though no standard send layout exists

Impact is SDK built send / quote_send transactions can omit required concrete accounts Routes may become unusable for users relying on generic account discovery. Note: If a project has generated SDK from its IDL, that SDK knows the real custom accounts, and that is safe in that case But the on chain metadata still publishes OftInfo.instructions.send = 1. So any generic OFT / OApp metadata driven resolver that does not use the project / app IDL SDK, and instead trusts OAppInfo / OftInfo, can still build the wrong account list

RECOMMENDATION

Set OFT instruction versions to 0, 1 is only correct if the framework actually defines OFT base instruction account layout v1, and currently It does not

RESOLUTION

Status: Resolved.

INFO I-15 ACKNOWLEDGED

Blacklist checks miss token account owners.

CATEGORY Logical Error LOCATION Transfer-hook REVIEW Console · Main Review

DESCRIPTION

During Token-2022 transfer-hook execution, the concrete transfer checks are described as covering "source/destination token-account PDAs plus signer/delegate" That omits the most important identities for blacklist enforcement `source_token_account.owner destination_token_account.owner` In SPL Token / Token-2022, the destination account passed to a transfer is a token account, not necessarily the user wallet. The wallet owner is stored inside the token account data. A user can create many token accounts, and those token account pubkeys will not equal the user wallet pubkey that was blacklisted Similarly, when a delegate transfers on behalf of a blacklisted user, the transfer authority / signing account is the delegate, not the token account owner. If only the delegate / signer is checked, a blacklisted owner can move funds through a clean delegate

So blacklist mode can be bypassed by any blacklisted user who creates new token accounts or uses a non blacklisted delegate, This defeats the stated policy that listed addresses are blocked

RECOMMENDATION

In the Token-2022 transfer_hook, deserialize the source and destination token accounts and enforce blacklist / whitelist policy against their owner fields. blacklist mode should reject if any of these identities are blacklisted `source_token_account.owner destination_token_account.owner transfer_authority_or_delegate` .

RESOLUTION

Status: Acknowledged.

The current token-account-based allowlist design was retained because wallet-owner checks would break account resolution for first-time recipient ATAs.

LOW L-01 RESOLVED

Quote Can Fail When Global Limits Are Disabled.

CATEGORY DoS

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L219-L227

REVIEW Console · Remediation Review 1

DESCRIPTION

Solana's `get_rate_limit_usages` follows the same high-level order as the EVM implementation: it resolves the effective rate-limit state/config, computes the current usage, and only then applies the global-disable override by returning maximum available capacity. On EVM, this ordering is safe because `rateLimits[id]` is a mapping lookup and always returns a default `RateLimit` struct even when the ID was never explicitly initialized.

On Solana, the same ordering has different behavior because per-EID rate limits are optional PDAs. When `use_global_state = false`, `get_rate_limit_usages` must load the per-EID PDA before it can compute usage. If that PDA is missing, the function returns `RateLimitNotInitialized` before reaching the global-disable branch, even when `is_globally_disabled = true`. Therefore, two implementations with the same logical ordering diverge because Solana has a missing-account failure mode that the EVM mapping model does not. This impacts more than the standalone rate-limit view.
Solana's `quote_oft` uses `get_rate_limit_usages` to compute `max_amount_ld`. Integrators that call `quote_oft` before `send` to obtain OFT limits, fee breakdown, and the expected receipt can fail unexpectedly even though `send` would succeed, because the mutating rate-limit path returns early when global rate limiting is disabled.

RECOMMENDATION

Make `get_rate_limit_usages` return maximum quote/view availability when global rate limiting is disabled before treating a missing optional per-EID PDA as an error.

RESOLUTION

Status: Resolved.

LOW L-02 RESOLVED

Rate-Limit No-Op Cases Still Require Per-EID PDA.

CATEGORY Unexpected Behavior

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L321-L355

REVIEW Console · Remediation Review 1

DESCRIPTION

Solana represents per-EID rate-limit state as an optional PDA, while EVM uses a mapping whose missing entries read as a zero-initialized struct. On Solana, `apply_rate_limit` returns early only when `is_globally_disabled` is true; for every other case it calls `get_effective_state_and_config_mut`, which returns `RateLimitNotInitialized` when `use_global_state = false` and the per-EID PDA is uninitialized. The two no-op gates the limiter actually honors, an effective direction disabled by the default config and an address-exempt user, are checked only after that resolution. So even when the config disables the relevant direction or the caller is exempt, a `send` or `lz_receive` for an EID whose per-EID `RateLimit` PDA was never created reverts with `RateLimitNotInitialized` before reaching the no-op gate. The read path has the same shape: `get_rate_limit_usages` resolves state and config first and only then applies the disabled-direction availability override, so `quote_oft` reverts for the same EIDs even though the effective answer is unlimited.

On EVM, `_getRateLimitStateAndConfig` reads `rateLimits[id]` directly from a mapping, so an uninitialized ID yields a zero state plus the default config and never reverts. The disabled-direction and address-exemption early returns are honored without any per-ID state ever being written. The Solana account model therefore introduces a missing-account failure mode for cases that EVM treats as unconditional no-ops. This is a Solana-only liveness footgun: the limiter is configured to allow the operation, but the call fails until the operator separately initializes per-EID state.

RECOMMENDATION

Evaluate the no-op conditions (disabled direction, address exemption) against the default config before treating a missing optional per-EID PDA as an error, or have the resolver fall back to default config and zeroed state to match EVM mapping semantics.

RESOLUTION

Status: Resolved.

LOW L-03 RESOLVED

BurnMint Whitelist-Mode Escrow Send DoS.

CATEGORY Compatibility

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/send.rs#L240

REVIEW Console · Remediation Review 1

DESCRIPTION

On a BurnMint OFT `send`, `console_oft` always routes the full amount from the sender to the escrow before burning. `lock_or_burn` issues `invoke_transfer_checked(token_source → token_escrow, amount_sent_ld)` and only then burns from the escrow. That `user → escrow` leg is a Token-2022 `transfer_checked`, so it fires the transfer hook with source = sender ATA, destination = escrow ATA, and authority = sender signer, and the hook runs the full source plus destination plus authority allowlist check. In Whitelist mode `AllowlistMode::allows` returns the whitelist flag, so the escrow ATA, as the transfer destination, must itself be whitelisted or the hook returns `DestinationBlocked` and the send reverts.

The escrow source-bypass does not rescue this leg. The bypass marker is keyed off the transfer source, but on send the escrow is the destination, not the source, which the hook comment in `transfer_hook.rs` confirms. The net effect is that enabling Whitelist mode silently bricks all BurnMint outbound sends until an operator separately whitelists the escrow token account. The condition is operator-introduced and recoverable by whitelisting the escrow, but it is easy to overlook because the requirement is undocumented.

This requirement has no EVM analogue. On EVM, BurnMint `_debit` burns the amount directly from the user via `burn(_from, amountSentLD)` with no intermediate vault, and `ERC20Plus.burn` checks only `onlyAllowlisted(_from)`; the fee is minted to `feeDeposit` via `mint()`, which carries no allowlist modifier. EVM BurnMint therefore needs only the user whitelisted, there is no escrow or vault to whitelist. The Solana-only escrow-routing design introduces the extra destination gate. The EVM-parity documentation does not surface this and in fact contradicts it: `transfer-hook-evm-parity-analysis.md` labels the BurnMint send path "Equivalent for OFT-controlled burn path," describing it only as "checks source before burn" and omitting the destination and authority checks that the same hook performs. An operator porting a BurnMint deployment from EVM, or a reviewer relying on the parity table, would have no indication that the escrow must be whitelisted, so every outbound `send` reverts with `DestinationBlocked` until the escrow is explicitly whitelisted.

RECOMMENDATION

Auto-whitelist the escrow token account when Whitelist mode is enabled. At minimum, document the requirement and correct the `transfer-hook-evm-parity-analysis.md` row so it no longer labels the BurnMint send path "Equivalent" without noting that the escrow destination check is a Solana-only gate absent from EVM `burn`.

RESOLUTION

Status: Resolved.

LOW L-04 ACKNOWLEDGED

Missing Mint Authority Check During Initialize.

CATEGORY Validation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/init_console_oft.rs#L52-L56

REVIEW Console · Remediation Review 1

DESCRIPTION

`init_console_oft` allows an OFTStore to be initialized with `oft_type = BurnMint` without validating that the OFTStore can actually mint the underlying token. The inbound `lz_receive` path later requires the token mint authority to be either the `oft_store` PDA itself or a compatible Token-2022 multisig with quorum 1 that includes `oft_store` as a signer. If the mint authority is an admin wallet, an incompatible multisig, or another authority the OFTStore cannot satisfy, initialization still succeeds but inbound delivery fails when the program attempts to mint.

This can leave a deployment in a difficult or permanent failure state. The setup is only recoverable while the current mint authority remains controllable and can be changed through the token program. If the mint authority is disabled or cannot be reassigned to a compatible authority, the BurnMint OFT instance remains unable to receive inbound messages.

RECOMMENDATION

During initialization of the BurnMint type, require proof that the mint authority is either `oft_store` or a compatible 1-of-n Token-2022 multisig containing `oft_store`

RESOLUTION

Status: Acknowledged.

LOW L-05 RESOLVED

Enforced options msg_type mismatch.

CATEGORY Configuration

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/helpers/mod.rs#L10-L14

REVIEW Console · Remediation Review 1

DESCRIPTION

The Solana OFT implementation defines `MsgType::Send = 0` and `MsgType::SendAndCall = 1`, while LayerZero's public FAQ and EVM OFT implementation use `msgType = 1` for `SEND` and `msgType = 2` for `SEND_AND_CALL`. The `send()` path derives the `EnforcedOptions` PDA from the local `msg_type()` result. If an operator follows the documented `1/2` convention or uses shared tooling that assumes EVM constants, they can configure `msgType = 1` for plain sends and `msgType = 2` for compose sends. Plain Solana sends then look for the unconfigured `msgType = 0` PDA; when it is missing, `try_load_account()` returns `None`, and `combine_options()` falls back to empty enforced options. As a result, the message can proceed using only caller-provided `extra_options`. Similarly, packets with compose messages will use the `SEND` options and the `SEND_AND_CALL` options will be unused.

This misconfiguration is also easier to introduce because `SetEnforcedOptionsParams::msg_type` accepts raw numeric values as its type is `u16`.

RECOMMENDATION

Either make the SVM values `1/2` as well, or explicitly document that on Solana `0/1` should be used and consider overriding `set_enforced_options` with additional validaiton.

RESOLUTION

Status: Resolved.

INFO I-01 RESOLVED

Transfer Hook Index-3 Authority Doc Mislabel.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/init_transfer_hook.rs#L127

REVIEW Console · Remediation Review 1

DESCRIPTION

The account-index documentation table in `init_transfer_hook.rs` labels index 3 as `authority / owner`. This is inaccurate: index 3 is the transfer authority that Token-2022 passes into the `Execute` hook, which may be the source owner, an approved delegate, or the mint's permanent delegate. The `owner` qualifier captures only one of the three cases and is misleading for the delegate and permanent-delegate recovery paths. The authoritative description in `transfer_hook.rs` correctly documents index 3 as `authority (source owner, approved delegate, or permanent delegate)`, so the two canonical index tables disagree with each other.

The runtime constant `AUTHORITY_ACCOUNT_INDEX` is correct; only the doc row is wrong, so there is no behavioral impact, but a reader relying on the `init` table could misreason about the PD fund-recovery and delegated-transfer flows that key the `authority_allowlist_entry` PDA off this account.

RECOMMENDATION

Update the index-3 row in the `init_transfer_hook.rs` table to match `transfer_hook.rs`. Use either `authority` alone or the fully-qualified `authority (owner / delegate / permanent delegate)` so the two tables are consistent and the delegate / permanent-delegate cases are not obscured.

RESOLUTION

Status: Resolved.

INFO I-02 RESOLVED

Document Solana Compose Codec Layout.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/codec/compose_msg.rs#L4-L23 and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

The Solana compose codec comment says it is aligned with `OFTComposeMsgCodec`, but the implementation encodes `amount_ld` as a `u64` occupying 8 bytes. Its compose header is therefore `[nonce:8][src_eid:4][amount_ld:8]`, with `compose_from` at offset 20 and user compose payload at offset 52. The EVM OFT compose codec encodes `amountLD` as `uint256` in `abi.encodePacked`, so the header is `[nonce:8][srcEid:4][amountLD:32]`, with `composeFrom` at offset 44 and user compose payload at offset 76. If the compact 8-byte Solana layout is intentional, the issue is the comment/documentation rather than the codec behavior. The current wording can make readers assume the Solana payload is byte-compatible with the EVM `OFTComposeMsgCodec` layout even though the offsets differ.

RECOMMENDATION

Update the codec comment and related docs to state that the Solana compose payload uses a Solana-specific compact layout with an 8-byte `amount_ld`, `COMPOSE_FROM_OFFSET = 20`, and `COMPOSE_MSG_OFFSET = 52`.

RESOLUTION

Status: Resolved.

INFO I-03

RESOLVED

Shared Program Upgrades Affect All OFT Instances.

CATEGORY Trust Assumptions

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/lib.rs#L37-L43

REVIEW Console · Remediation Review 1

DESCRIPTION

Solana `console_oft` can host multiple OFTStore instances under one deployed program ID. Although each instance has separate state and admin configuration, all instances execute the same program binary. If that program remains upgradeable, a later upgrade changes behavior for existing and future OFTStore instances. This differs from EVM, where developers deploy separate OFT proxy instances from the LayerZero implementation contracts. Upgrading one EVM proxy does not automatically affect other OFTs, while upgrading a shared Solana program affects every OFTStore under that program ID. This is an implicit trust assumption for developers using a shared deployment.

RECOMMENDATION

Document the shared upgrade-authority trust model and advise production deployers to self-deploy, use governance-controlled upgrades, or freeze the program when appropriate.

RESOLUTION

Status: Resolved.

INFO I-04 RESOLVED

Dust Can Misalign Hook Accounts.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/send.rs#L284-L290

REVIEW Console · Remediation Review 1

DESCRIPTION

`send` decides whether to run the escrow → fee deposit transfer from `fee_ld = amount_sent_ld - amount_received_ld`, not from the configured fee bps. Since `amount_received_ld` is rounded down by `remove_dust`, `fee_ld` includes both configured OFT fee and the dust. A send with zero fee bps can therefore still enter `collect_fee` when `amount_ld` is not aligned to `ld2sd_rate`.

For Token-2022 mints with an active transfer hook, `collect_fee` expects a second hook-account prefix before the Endpoint accounts. Callers that assemble `remaining_accounts` from the configured OFT fee can omit this second hook prefix for dust-only fees, causing the program to parse the Endpoint account as the fee hook program and revert with `InvalidTransferHookProgram`. The repo's test helper also reflects this assumption by computing `hasFee` from `amount * bps / 10000`, while the on-chain condition is the dust-inclusive receipt delta. `// Fee hook accounts are only included when fees will be collected (fee_ld > 0). // The on-chain program splits remaining_accounts as: // [transfer_hook..., fee_hook...(if fee>0), endpoint...] // so including fee hooks when fee=0 would misalign the endpoint accounts slice.

// Must compute fee_ld using the same integer arithmetic as on-chain (amount * bps / 10000).`

RECOMMENDATION

Document that fee-hook accounts must be included whenever `amount_sent_ld - amount_received_ld > 0`, including dust-only deltas, not only when configured fee bps produces a nonzero fee.

RESOLUTION

Status: Resolved.

INFO I-05 RESOLVED

RoleType is always required.

CATEGORY Informational LOCATION apps/oft-app/contracts/solana/oft/macros/src/oft.rs#L49

REVIEW Console · Remediation Review 1

DESCRIPTION

The [I-06] Undocumented `RoleType` requirement in `#[oft]` was resolved with the following comment that says: #[oft] now [...] accepts an optional role_type = attribute, defaulting to crate::RoleType when omitted The actual implementation doesn't have such fallback logic, instead the `role_type` is required.

RECOMMENDATION

Confirm whether the field should be required or optional.

RESOLUTION

Status: Resolved.

INFO I-06 RESOLVED

Receive-Types Token Mint Comment Mismatch.

CATEGORY Informational

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive_types_info.rs and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

The step-1 receive-types comment says `token_mint` is included because V2 needs it to "derive token_program for ATA creation". The implementation uses the token program stored in `oft_store.token_program`, not a token program derived from `token_mint`. `lz_receive_types_info` derives the destination ATA with `ctx.accounts.oft_store.token_program` and passes that stored token program as its own required account. `lz_receive_types_v2` then receives `token_program` as a separate account and uses it in the token-mint constraint and destination ATA derivation. The `token_mint` account is still needed by V2, but for mint validation and mint-data reads such as mint authority, decimals, and transfer-hook extension metadata. The current comment points readers to the wrong dependency when reviewing receive-account planning.

RECOMMENDATION

Update the comment to explain that `token_mint` is included so V2 can validate and read the mint, including transfer-hook metadata, while `token_program` comes from `oft_store.token_program`.

RESOLUTION

Status: Resolved.

INFO I-07 RESOLVED

Mint Bypass Comment Names Wrong Modifier.

CATEGORY Informational

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs#L16-L20

REVIEW Console · Remediation Review 1

DESCRIPTION

The transfer-hook module comment says EVM `mint()` has neither `whenNotGloballyPaused` nor `onlyAllowlisted` modifiers. In the scoped EVM implementation, the pause modifier used by `ERC20Plus` is `whenNotPaused`; `mint()` is gated only by `onlyRole(MINTER_ROLE)`, while `burn`, `transfer`, and `transferFrom` use `whenNotPaused` and/or allowlist modifiers. The high-level statement that `mint()` bypasses pause/list checks is correct, but the modifier name does not match the EVM contract. This makes the Solana comment harder to verify against the referenced EVM behavior.

RECOMMENDATION

Change the comment to say that EVM `mint()` has neither `whenNotPaused` nor `onlyAllowlisted` modifiers.

RESOLUTION

Status: Resolved.

INFO I-08 RESOLVED

Fees Should Account For Executor Funded ATA Creation.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive.rs#L80-L81

REVIEW Console · Remediation Review 1

DESCRIPTION

`lz_receive` creates the recipient token account with `init_if_needed` and uses the executor as payer for rent. The recipient address is taken from the delivered OFT message, so messages to first-time recipients can require the executor to fund a new ATA during destination execution. `quote_send` quotes the LayerZero endpoint fee from the message and options, but the on-chain quote path does not itself check whether the destination ATA already exists or whether rent will be needed. Integrators should ensure that the source-chain quoted/executor fees and execution options are sufficient to cover destination-side ATA creation and rent when the recipient token account may not already exist.

RECOMMENDATION

Document that first-time recipient ATA creation is funded during `lz_receive` by the executor payer, and that source-side fee quoting/options should account for this destination rent cost.

RESOLUTION

Status: Resolved.

INFO I-09 RESOLVED

Document Escrow Entries In Whitelist Mode.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs#L165-L187

REVIEW Console · Remediation Review 1

DESCRIPTION

In whitelist mode, an OFT escrow needs both a whitelist entry and a source-bypass entry for the intended bridge flow. Outbound sends transfer from the user to the escrow, so the escrow is checked as the destination and must be whitelisted. Inbound receives transfer from the escrow to the recipient, so the escrow is the source and needs source-bypass to avoid requiring every recipient or authority to be whitelisted. The `PermanentDelegate` recovery path follows EVM parity and determines recovery eligibility from allowlist status, not source-bypass status. If the escrow is de-whitelisted while source-bypass remains enabled, the `PermanentDelegate` can move funds from the escrow through the recovery path, even though source-bypass still applies to normal escrow-as-source transfers. OFT admins should be aware of the misconfiguration risk of source-bypassed but de-whitelisted escrows.

RECOMMENDATION

Document that whitelist-mode OFT escrows require both whitelist and source-bypass entries, and the risk of de-whitelisting a source-bypassed escrow.

RESOLUTION

Status: Resolved.

INFO I-10 RESOLVED

QuoteOFT Max Understates Sendable Amount.

CATEGORY Unexpected Behavior

LOCATION console_oft/src/instructions/quote_oft.rs(QuoteOFT::apply) and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

`quote_oft` derives `oft_limit.max_amount_ld` by taking the outbound rate-limit availability and passing it through `get_amount_before_fee` : `// quote_oft.rs if max_amount_ld ≠ 0 && max_amount_ld ≠ u64::MAX { max_amount_ld = fee_config::get_amount_before_fee( ctx.accounts.oft_store.default_fee_bps, fee_config.as_ref(), max_amount_ld, ); } let oft_limit = OFTLimit { min_amount_ld: 0, max_amount_ld };` `get_amount_before_fee` inverts only the fee and does nothing about dust: `// fee_config_view.rs pub fn get_amount_before_fee(default_fee_bps: u16, cfg: Option<&FeeConfig>, amount_after_fee: u64) -> u64 { let bps = get_fee_bps(default_fee_bps, cfg) as u128; let denom = MAX_FEE_BASIS_POINTS as u128; if bps >= denom { return 0; } if bps == 0 { return amount_after_fee; } let amount_before_fee = (amount_after_fee as u128) * denom / (denom - bps); u64::try_from(amount_before_fee).unwrap_or(u64::MAX)

} `The actual send path instead applies dust removal after the fee, and the rate limiter is charged against the dust-removed received amount:` // send.rs (debit_view) let oft_fee = fee_config::get_oft_fee(oft_store.default_fee_bps, fee_config, amount_ld); let amount_received_ld = oft_store.remove_dust(amount_ld - oft_fee); // remove_dust(x) = x - x % ld2sd_rate `Because` remove_dust `maps a contiguous band of pre-fee amounts onto the same post-fee, dust-aligned received amount, several distinct pre-fee` amount_ld `values all produce the identical` amount_received_ld `. The fee-only inverse returns the smallest such pre-fee amount, so` max_amount_ld`lands one dust plateau short of the true maximum that fits within capacity.

The quote can therefore simultaneously report (a) that a specific requested send receives an amount within capacity and executes successfully, and (b) a `max_amount_ld` lower than that same requested amount. The under-report is always in the conservative direction, since the quote never advertises a max larger than what the send path accepts, so no rate-limit ceiling can be bypassed through it. Clients that trust `oft_limit.max_amount_ld` as a hard cap can reject sends that the program would accept, degrading the quote and view API's usefulness at the dust margin near the rate-limit ceiling. This is also the behavior on EVM, so if the decision is to fix this, it should also be fixed on EVM,

RECOMMENDATION

When converting post-fee capacity back to a max pre-fee send amount, account for the dust plateau by returning the largest pre-fee `amount_ld` whose `remove_dust(amount_ld - fee(amount_ld))` still fits within capacity, rather than the fee-only inverse. Or clearly document this mismatch and acknowledge it as intended.

RESOLUTION

Status: Resolved.

INFO I-11 RESOLVED

recoverFunds Parity Comment Uses Wrong Signature.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs

REVIEW Console · Remediation Review 1

DESCRIPTION

The transfer-hook doc comments cite the EVM parity as `recoverFunds(from, amount)`. The actual EVM signature is `recoverFunds(address _from, address _to, uint256 _amount)` (`ERC20Plus.sol#L144`), the `_to` recipient argument is omitted.

RECOMMENDATION

Update the comment to `recoverFunds(from, to, amount)`.

RESOLUTION

Status: Resolved.

INFO I-12 RESOLVED

Fund-Recovery Authority Rotation Undocumented.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_transfer_hook/src/instructions/transfer_hook.rs

REVIEW Console · Remediation Review 1

DESCRIPTION

Blocked-source fund recovery is gated solely on the mint's Token-2022 PermanentDelegate (`get_permanent_delegate(&mint_data) == Some(signer)` in `try_bypass_for_fund_recovery`), decoupled from the hook's RBAC. The docs already disclose this and label it intentional ("authority-model adaptation", "not bugs") in `transfer-hook-evm-parity-analysis.md`, `transfer-hook.md`, `AUDIT.md`, and the `transfer_hook.rs` module comment. Two consequences are NOT documented: (1) the PermanentDelegate rotates via a one-step SPL `SetAuthority` call with no pending/accept step, unlike EVM's two-step `DEFAULT_ADMIN_ROLE` handoff; (2) recovery keys off a principal distinct from the hook's DefaultAdmin, so it sits outside the governance protecting every other hook action. The capability is EVM-equivalent, so this is purely a documentation gap, not a privilege bug.

RECOMMENDATION

Note in `transfer-hook-evm-parity-analysis.md` that the recovery authority is the mint PermanentDelegate (a distinct principal from the hook DefaultAdmin) and rotates one-step via `SetAuthority` with no accept step.

RESOLUTION

Status: Resolved.

INFO I-13 RESOLVED

Rate Exemption Seed Name Typo.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L35 and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

The `RateLimitAddressExemption` PDA comment documents the seed as `RATE_LIMITER_EXEMPTION_SEED`. No constant with that name exists in the codebase. The actual seed constant is `RATE_LIMIT_EXEMPTION_SEED` in `seeds.rs`, and `RateLimitAddressExemption::seeds` uses that constant. The seed bytes are correct at runtime, but the comment points integrators and reviewers to a non-existent symbol.

RECOMMENDATION

Update the `RateLimitAddressExemption` PDA comment to reference `RATE_LIMIT_EXEMPTION_SEED`.

RESOLUTION

Status: Resolved.

INFO I-14 RESOLVED

Per-ID PDA Seed Width Docs.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L16 and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

The `RateLimit`, `PauseConfig`, and `FeeConfig` PDA comments document the id seed as `&eid.to_be_bytes()`, which suggests the 4-byte big-endian EID encoding. The actual seed helpers use `id.to_u128_be_bytes()`, producing a 16-byte big-endian `u128` seed for rate-limit, pause, and fee PDAs. An SDK author or reviewer copying the comments literally would derive different non-resolving PDA addresses.

RECOMMENDATION

Update the three PDA seed comments to state that the id component is the canonical 16-byte big-endian `u128` encoding, matching `id.to_u128_be_bytes()`.

RESOLUTION

Status: Resolved.

INFO I-15 RESOLVED

Rate Limiter Docs Cite Wrong File.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit

REVIEW Console · Remediation Review 1

DESCRIPTION

Several rate-limiter comments cite `RateLimiterCoreUpgradeable.sol` as the EVM reference for `_getRateLimitUsage`, `_getRateLimitUsages`, and `_getRateLimitStateAndConfig`. That file does not exist in the scoped EVM sources. The referenced functions live in `RateLimiterBaseUpgradeable.sol`, which the same Solana file already cites correctly elsewhere.

RECOMMENDATION

Replace the `RateLimiterCoreUpgradeable.sol` references with `RateLimiterBaseUpgradeable.sol`.

RESOLUTION

Status: Resolved.

INFO I-16 RESOLVED

Pause Docs Cite Wrong EVM Contract.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/pause/set_paused.rs#L5 and related helpers

REVIEW Console · Remediation Review 1

DESCRIPTION

The `console_oft` per-ID pause instruction comments describe EVM alignment as `PauseRBACUpgradeable.setPaused(...)` and `PauseRBACUpgradeable.setDefaultPaused(bool)`. Those functions are part of the per-ID pause implementation, `PauseByIDRBACUpgradeable`, while `PauseRBACUpgradeable` exposes only the global `pause` / `unpause` flow. The comments therefore point reviewers to the wrong EVM contract for the `IPauseByID` mapping. In addition, the README states that the `lz_receive` path is performing a pause check, which is not true.

RECOMMENDATION

Change the comments to reference `PauseByIDRBACUpgradeable.setPaused(...)` and `PauseByIDRBACUpgradeable.setDefaultPaused(bool)`. Also fix the readme.

RESOLUTION

Status: Resolved.

INFO I-17 RESOLVED

Misleading fee inverse edge cases.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/fee_config/fee_config_view.rs#L69-L84

REVIEW Console · Remediation Review 1

DESCRIPTION

`get_amount_before_fee()` is documented as the inverse of the fee calculation: given a post-fee amount, it returns the pre-fee amount needed. This description is incomplete for edge cases where the inverse is not unique or cannot be represented exactly. When `amount_after_fee == 0`, multiple pre-fee amounts can produce zero post-fee output depending on the configured fee and integer rounding. This is especially visible when `bps == MAX_FEE_BASIS_POINTS`, where every pre-fee amount produces zero post-fee output, so returning `0` is only one conservative representative value rather than a uniquely inferred pre-fee amount.

The helper also clamps results that exceed `u64::MAX` through `u64::try_from(amount_before_fee).unwrap_or(u64::MAX)`, which means the returned value may be a saturated cap rather than the exact mathematical pre-fee amount. These behaviors may be acceptable for quote-limit usage, but the current "inverse of fee" documentation does not describe them.

RECOMMENDATION

Update the documentation for `get_amount_before_fee()` to describe its edge-case behavior explicitly. State that `0` is returned as a conservative representative when the requested post-fee amount is zero or when a 100% fee makes every pre-fee amount produce zero post-fee output, and that values above `u64::MAX` are saturated to `u64::MAX`.

RESOLUTION

Status: Resolved.

INFO I-18 ACKNOWLEDGED

Arbitrary compose plans expose Executor tokens.

CATEGORY Validation LOCATION send_compose_if_needed() REVIEW Console · Remediation Review 1

DESCRIPTION

OFT sender can cause the console_oft receiver to create a compose job for an attacker controlled composer, and that composer V2 planner can request Executor signer placeholders in the final transaction On the compose side, the framework accepts arbitrary V2 planner output And each account can be a concrete pubkey, ALT entry, Executor payer, additional Executor signer, or context account There is no framework level rule that says: Payer may only be used in the known rent payer position, or Signer(_) may only be used for a specific predeclared account init pattern That means a malicious composer can return a plan that causes the Executor to place its own hot wallet / payer signer into attacker chosen instruction accounts Attack flow is 1) The attacker deploys a malicious Solana composer program with a composer account, for example, malicious_composer_store

2) The attacker sends OFT cross-chain with `send_to = malicious_composer_store` `compose_msg = attacker-controlled payload` 3) The honest console_oft::lz_receive validates the LayerZero peer and clears the verified payload 4) Because the payload contains compose data, console_oft calls Endpoint `send_compose` with to: malicious_composer_store 5) The Executor later discovers and executes the compose job. It asks the malicious composer for lz_compose_types_v2 6) The malicious composer returns a plan that includes the Executor payer or signer as a writable signer account. 7) The Executor signs and sends the real compose transaction 8) Inside lz_compose, the malicious composer uses the Executor payer signature to call Token-2022 `transfer_checked( from = executor token account, to = attacker token account, authority = executor payer )`

9) Because the Executor payer really signed the transaction, Token-2022 accepts the transfer This is not a malicious OApp only scenario. The console_oft program is the one creating the compose job, and the attacker chooses the compose target through a normal user facing OFT message The next impact is mitigated: "the attacker can drain SOL up to the execution fee limit" The preExecute/postExecute checks cap the payer's balance loss, and the executor is already reimbursed for the amount spent through the fees paid upfront by the user, so this does not harm the executor The only imapct Is, Theft of non SOL assets if the Executor wallet also owns SPL token accounts The malicious lz_compose handler can then CPI into the SPL Token program and transfer tokens from the executor's token account we might need to downgrade this, If LayerZero executor wallets will always have SOL only

but we didn't find this enforced anywhere in the code / sdk

PoC `corepack enable corepack pnpm install --frozen-lockfile cd apps/project-types/console-oft-app/contracts/solana corepack pnpm test:poc:native`

INFO I-18 CONTINUED 2/2 ACKNOWLEDGED

Arbitrary compose plans expose Executor tokens.

RECOMMENDATION

Fix in the Executor side after ALT resolution, before building the final compose transaction. Apply a guard in compose-types-v2.ts, inside `buildLzComposeExecutionPlan` , Create a helper function like assertSafeComposePlan(), and call it `const tables = await fetchAllAddressLookupTable({ rpc }, resp.alts) assertSafeComposePlan(executorProgram, tables, resp, payer)`

RESOLUTION

Status: Acknowledged.

LOW L-01 RESOLVED

Missing RateLimit PDA mismatch.

CATEGORY Logical Error LOCATION rate_limit.rs REVIEW Console · Remediation Review 2

DESCRIPTION

The `apply_rate_limit()` function allows both `use_global_state == false` and missing per eid `RateLimit` PDA, when the following conditions are true. `// EVM parity: same early return gates as _applyRateLimit . if (!forward_enabled && (!backward_enabled || !net_accounting_enabled)) || (address_exemption_enabled && is_address_exempt) { return Ok(()); }` This can happen when: both directions are disabled current direction and net accounting are disabled the address is exempted In this case the send flow will be successfully executed.

The third condition, the address exemption, is not present in `get_outbound_available_for_quote()` and `get_rate_limit_usages()` because there is no user provided. Even though the documentation claim `A missing per-EID PDA is allowed here only when the real send/outflow path would also return before touching state.` is technically not violated, this creates a mismatch between `quote_oft()/get_rate_limit_usages()` and `send()` where a user quoting would see failure, while he can successfully execute the send. The same issue is present on EVM, but there the quote will instead return stale value instead of failing.

RECOMMENDATION

Either document that the functions are unreliable in these specific cases or consider making the two paths behave the same way, for example don't include the exemption check when deciding whether to allow empty eid `RateLimit` PDA

RESOLUTION

Status: Resolved.

INFO I-01 RESOLVED

Informational Note Regarding Rate Limit Test.

CATEGORY Informational LOCATION console_oft/src/state/rate_limit.rs#L760

REVIEW Console · Remediation Review 2

DESCRIPTION

`test_missing_per_eid_state_quote_errors_when_net_accounting_needs_state` is intended to validate the quote-specific path for missing per-EID rate-limit state when outbound is disabled but inbound net accounting still requires state. However, the test calls `get_rate_limit_usages` instead of `get_outbound_available_for_quote` and does not cover intended quote functionality. Noting that this is only a test-suite issue and does not affect production behavior.

RECOMMENDATION

Update the test to call `get_outbound_available_for_quote` directly so the assertion covers the intended quote functionality.

RESOLUTION

Status: Resolved.

INFO I-02 RESOLVED

Rate-limit PDA requirement is underdocumented.

CATEGORY Documentation LOCATION README.md REVIEW Console · Remediation Review 2

DESCRIPTION

The documentation states that default-and-override behavior is the same on both platforms, but this omits an important Solana account-model requirement for `RateLimit` state. On EVM, an uninitialized `rateLimits[id]` mapping entry behaves like a zeroed per-ID state while falling back to the default config. So if `useGlobalStateFlag == false && isGloballyDisabledFlag` and the rest of the preconditions about net accounting and rent exemption are not met, the code will use the empty state with the default config. On SVM, when `use_global_state` is `false` and the per-EID `RateLimit` PDA is missing, mutating paths such as `send()` and `lz_receive()` can return `RateLimitNotInitialized` if state must be updated.

RECOMMENDATION

Document this EVM/SVM difference

RESOLUTION

Status: Resolved.

INFO I-03 RESOLVED

Misleading Globally-Disabled Usage Doc.

CATEGORY Documentation

LOCATION apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/state/rate_limit.rs#L237-L238 and related helpers

REVIEW Console · Remediation Review 2

DESCRIPTION

The documentation on `get_rate_limit_usages` states that when rate limiting is globally disabled, "both availabilities are `u64::MAX`, but usage values still reflect decayed on-chain state" (rate_limit.rs#L237-L238). This is asserted unconditionally, but the function has a branch that contradicts it. When the resolved state is absent, `get_rate_limit_usages` takes the missing-state path (rate_limit.rs#L250-L257): `let Some(state) = state else { require!( oft_store.is_globally_disabled || (config.outbound_skips_state() && config.inbound_skips_state()), OFTError::RateLimitNotInitialized ); return Ok(RateLimitUsages::unlimited()); };` `RateLimitUsages::unlimited()` hardcodes `outbound_usage: 0` and `inbound_usage: 0` (rate_limit.rs#L148-L155) and reads no on-chain state.

As a result, for the same logical configuration (globally disabled), the reported usage differs depending on whether the optional per-EID rate-limit account exists: when it exists, usage is the decayed on-chain value the comment promises; when it is absent, usage is a hardcoded zero returned without touching state. The doc sentence describes only the first case. There is no behavioral defect: a missing per-EID account has never recorded any in-flight amount, so its correctly-decayed usage genuinely is zero, and returning zero is consistent with the EVM mapping model where an uninitialized `id` reads as a zero-usage entry. The impact is confined to documentation precision. An integrator or reviewer relying on the comment could assume the globally-disabled usages view always performs a decay computation over stored state, when the missing-account branch instead returns fixed zeros.

RECOMMENDATION

Update the docs to reflect the actual behavior, qualifying the sentence so it applies only when state exists.

RESOLUTION

Status: Resolved.

MEDIUM **M-01** RESOLVED

init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one

CATEGORY	Compatibility
LOCATION	apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/init_console_oft.rs:InitConsoleOFT.oft_escrow, apps/project-types/console-oft-app/contracts/solana/Cargo.lock (anchor-spl 1.1.2 -> spl-token-2022-interface 2.1.0)
REVIEW	Main Review

DESCRIPTION

`init_console_oft` creates `oft_escrow` with Anchor's `init`, and for a Token-2022 mint Anchor's generated code sizes the account by listing the mint's extensions through `anchor-spl`'s re-exported `spl-token-2022-interface` 2.1.0. That version's [`ExtensionType`](https://docs.rs/spl-token-2022-interface/2.1.0/spl_token_2022_interface/extension/enum.ExtensionType.html) stops at 27 and maps unknown types to `InvalidAccountData`. PermissionedBurn is type 28, added in [Token-2022 v11](https://github.com/solana-program/token-2022/releases/tag/program%40v11.0.0) and present only in [interface 3.x](https://docs.rs/spl-token-2022-interface/3.1.1/spl_token_2022_interface/extension/enum.ExtensionType.html), which the program itself uses. Every `anchor-spl` release, including [1.2.0](https://docs.rs/crate/anchor-spl/1.2.0/source/Cargo.toml.orig) and [master](https://github.com/solana-foundation/anchor/blob/master/spl/Cargo.toml), still requires interface `"2"`, the same kind of lag Anchor hit in [anchor#2801](https://github.com/solana-foundation/anchor/issues/2801).

```rust
#[account(
init,
payer = payer,
seeds = OFTStore::oft_escrow_seeds(&oft_store.key()),
bump,
token::authority = oft_store,
token::mint = token_mint
)]
pub oft_escrow: Box<InterfaceAccount<'info, TokenAccount>>,
```

As a result, `init_console_oft` fails with `InvalidAccountData` for every PermissionedBurn mint in both custody modes and rolls back. A Mollusk test that runs the real `console_oft`, Token-2022 v11 and Endpoint programs returns `Failure(0, InvalidAccountData)` for a

M-01 init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one

CONTINUED

PermissionedBurn mint and `Success` for plain and transfer-hook mints. `init_console_oft` is the only way to create an `OFTStore`, so the PermissionedBurn setup this release adds (`set_token_manager` and the PermissionedBurn branch of `Send::burn_from_escrow`) cannot be deployed.

``rust
// tests/integration/poc_pb_init.rs (add `mod poc_pb_init;` to tests/integration/main.rs)
// cargo test -p console-oft-integration-tests --locked poc_ -- --nocapture --test-threads=1
use crate::harness::*;
use console_oft::state::RoleType as OftRole;
use spl_token_2022_interface::extension::BaseStateWithExtensions;

fn oapp_registry(oapp: Pubkey) -> Pubkey {
Pubkey::find_program_address(&[b"OApp", oapp.as_ref()], &host(endpoint::ID)).0
}

fn init_ix(h: &Harness, initializer: Pubkey, oft_type: ::oft::types::OftType) -> Instruction {
let store = Pubkey::find_program_address(&[b"OFT", initializer.as_ref()], &program_id()).0;
let endpoint_id = host(endpoint::ID);
let mut instruction = ix(
console_oft::ID,
console_oft::accounts::InitConsoleOFT {
payer: to_program_pubkey(&h.f.payer),
initializer: to_program_pubkey(&initializer),
oft_store: to_program_pubkey(&store),
default_rate_limit: to_program_pubkey(&rate_limit_pda(&store, 0)),
token_mint: to_program_pubkey(&h.f.token_mint),
oft_escrow: to_program_pubkey(&oft_escrow_pda(&store)),
fee_deposit: to_program_pubkey(&h.f.fee_deposit),
token_program: to_program_pubkey(&h.token_program),
system_program: Default::default(),
admin_role_member: to_program_pubkey(&role_member_pda(
&store,
OftRole::DefaultAdmin,
&h.f.admin,
)),
event_authority: to_program_pubkey(&event(program_id())),
program: console_oft::ID,

M-01 init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one

CONTINUED

```
},
console_oft::instruction::InitConsoleOft {
params: console_oft::instructions::InitConsoleOFTParams {
oft_type,
admin: to_program_pubkey(&h.f.admin),
shared_decimals: TOKEN_DECIMALS - 2,
},
},
);
instruction.accounts.extend([
AccountMeta::new_readonly(endpoint_id, false),
AccountMeta::new(h.f.payer, true),
AccountMeta::new_readonly(store, false),
AccountMeta::new(oapp_registry(store), false),
AccountMeta::new_readonly(Pubkey::default(), false),
AccountMeta::new_readonly(event(endpoint_id), false),
AccountMeta::new_readonly(endpoint_id, false),
]);
instruction
}

fn fresh_initializer(h: &Harness) -> (Pubkey, Pubkey) {
let initializer = Pubkey::new_unique();
h.f.insert(initializer, Account::new(100_000_000, 0, &SYSTEM_PROGRAM));
let store = Pubkey::find_program_address(&[b"OFT", initializer.as_ref()], &program_id()).0;
(initializer, store)
}

#[test]
fn poc_init_console_oft_fails_for_permissioned_burn_mint() {
for (label, oft_type) in [
("BurnMint", ::oft::types::OftType::BurnMint),
("LockUnlock", ::oft::types::OftType::LockUnlock),
] {
let h = Harness::with_pb(false, false, true);
let mint = h.account(h.f.token_mint);
let exts = StateWithExtensions::<Mint>::unpack(&mint.data)
.unwrap()
.get_extension_types()
```

M-01 init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one

CONTINUED

```
.unwrap();
assert!(exts.contains(&ExtensionType::PermissionedBurn));

let (initializer, store) = fresh_initializer(&h);
let result = h.run(&[init_ix(&h, initializer, oft_type)]);
println!("{label} PB mint init result: {:?}", result.program_result);
assert_eq!(
result.program_result,
TransactionProgramResult::Failure(0, ProgramError::InvalidAccountData),
);
assert!(h.account(store).data.is_empty());
assert!(h.account(oft_escrow_pda(&store)).data.is_empty());
}
}

#[test]
fn poc_control_init_succeeds_without_permissioned_burn() {
for hook in [false, true] {
let h = Harness::with_pb(false, hook, false);
let (initializer, store) = fresh_initializer(&h);
let result = h.run(&[init_ix(&h, initializer, ::oft::types::OftType::BurnMint)]);
println!("hook={hook} non-PB mint init result: {:?}", result.program_result);
assert!(result.program_result.is_ok());
assert_eq!(h.token(oft_escrow_pda(&store)).owner, store);
}
}

// Output:
// hook=false non-PB mint init result: Success
// hook=true non-PB mint init result: Success
// BurnMint PB mint init result: Failure(0, InvalidAccountData)
// LockUnlock PB mint init result: Failure(0, InvalidAccountData)
// test result: ok. 2 passed; 0 failed
```

RECOMMENDATION

Create `oft_escrow` without Anchor's extension-aware sizing: size it with the program's own `spl-token-2022-interface` 3.1.1, create the PDA through the System program with the escrow seeds, and CPI `InitializeAccount3` with `oft_store` as owner.

M-01 init_console_oft reverts for every PermissionedBurn mint, so no console OFT can be deployed for one — CONTINUED

RESOLUTION

Status: Resolved.

LOW **L-01** RESOLVED

A recipient can close its empty frozen ATA to receive a credit the docs say is redirected

CATEGORY	Validation
LOCATION	apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive.rs:apply() apps/project-types/console-oft-app/contracts/solana/docs/usage.md apps/project-types/console-oft-app/contracts/solana/docs/solana-evm-parity-analysis.md
REVIEW	Main Review

DESCRIPTION

`docs/usage.md` lists a frozen recipient on a mint that is not on Token ACL as a redirect trigger:

> the recipient's ATA is frozen and cannot be thawed (no policy selected, permissionless thaw disabled, or the mint not on Token ACL)

and parity doc D7 does the same: "frozen and not thawable (funded, or mint not ACL-governed) → redirect to escrow".

`lz_receive` creates `token_dest` with `init_if_needed` and only then reads its frozen bit:

```rust
let frozen = ctx.accounts.token_dest.is_frozen();

let credit_to = match (frozen || policy_blocked, redirect_escrow) {
(false, _) => ctx.accounts.token_dest.to_account_info(),
(true, Some(escrow)) => escrow,
(true, None) => return err!(OFTError::CreditBlocked),
};
```

Token-2022 `process_close_account` does not check the frozen bit, only the balance:

```rust
if !source_account.base.is_native() && u64::from(source_account.base.amount) != 0 {
return Err(TokenError::NonNativeHasBalance.into());
}
```

L-01 A recipient can close its empty frozen ATA to receive a credit the docs say is redirected CONTINUED

The owner of an empty frozen ATA can therefore close it and call `lz_receive`, which is permissionless, in the same transaction. Unless the mint has DefaultAccountState set to Frozen, the recreated ATA starts unfrozen, `frozen` is false, and the full credit goes to the recipient instead of the escrow, with no `CreditRedirected` event. `init_console_oft` does not require DefaultAccountState, so this applies to any plain Token-2022 mint with a freeze authority. The recipient also reclaims the ATA rent, which the executor pays again.

The token-manager usage doc and parity D4 say a freeze on an empty account is not a durable hold and that the policy is the tool for one, but only in the context of Token ACL's permissionless thaw. The console docs present the non-ACL case as a working hold.

A funded frozen ATA cannot be closed, and a policy-blocked recipient is not affected, because `policy_blocked` is read from the policy record, not the account. The credit redirect path is new in this diff.

RECOMMENDATION

Correct `docs/usage.md` and D7 to say that a freeze on an empty account is not a durable hold on any mint, and that the pubkey policy is the way to hold a recipient's credits. If the non-ACL freeze hold is meant to work, require a pubkey policy whenever redirect is configured, or reject a mint in `init_console_oft` that has a freeze authority but no DefaultAccountState Frozen.

RESOLUTION

Status: Resolved.

LOW L-03 RESOLVED

Confidential balances cannot be recovered or burned by pd_recover and pd_burn

CATEGORY Logical Error

LOCATION apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/permanent_delegate/pd_recover.rs:apply(), apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/permanent_delegate/pd_burn.rs:apply(), apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/root/init_token_manager.rs, apps/project-types/console-oft-app/contracts/solana/docs/solana-evm-parity-analysis.md (D9)

REVIEW Main Review

DESCRIPTION

`pd_recover` and `pd_burn` act as the Permanent Delegate through `transfer_checked` and `burn_checked`, which only debit the source's public balance. Token-2022 gives the Permanent Delegate no authority over confidential balances: every confidential instruction validates the account owner only. `init_token_manager` accepts mints with `ConfidentialTransferMint`, and the token-manager explicitly handles confidential accounts in its thaw emptiness checks (`interface/src/acl/thaw.rs:44-48`, README:248). On such a mint a holder can call the owner-only confidential `Deposit`, which needs no proof and runs no transfer hook, so it succeeds before any block and also after a Blacklister blocks the owner, until a Freezer freezes the account.

```rust
spl_token_2022::onchain::invoke_transfer_checked(
&a.token_program.key(),
a.source.to_account_info(), // debits the public amount only
a.mint.to_account_info(),
a.destination.to_account_info(),
a.pd_signer.to_account_info(), // Permanent Delegate
ctx.remaining_accounts,
amount,
a.mint.decimals,
&[&[SIGNER_SEED, state.as_ref(), &[SignerKind::PermanentDelegate as u8], &
[ctx.bumps.pd_signer]]],
)?;
```

Once the account is frozen, `pd_recover` fails with `InsufficientFunds` for any amount above the public residue, and `pd_burn`, which does not thaw, burns at most the public balance even after a Thawer reopens the account. The shielded amount stays in the frozen account indefinitely: no issuer role can move or burn it, it remains in the mint supply, and the

L-03 Confidential balances cannot be recovered or burned by pd_recover and pd_burn — CONTINUED

account cannot be closed while its confidential balance is nonzero. The holder recovers everything if the account is later unblocked and thawed. EVM `ERC20Plus.recoverFunds` moves any amount of a non-allowlisted holder's balance (`ERC20Plus.sol:146-149`), yet parity D9 states that `pd_recover` matches `recoverFunds` and names only the frozen versus blocked trigger as the divergence. An issuer relying on recovery for restitution or sanctions enforcement therefore cannot seize value a holder shielded in time.

RECOMMENDATION

Document in the README and in parity D9 that confidential balances are outside `pd_recover` and `pd_burn`, and that issuers who need full recovery should either not enable confidential transfers or set `auto_approve_new_accounts = false` and approve only trusted accounts, since `Deposit` requires an approved account. If confidential mints are not meant to be supported, `init_token_manager` can instead reject `ConfidentialTransferMint` with `UnsupportedExtension`.

RESOLUTION

Status: Resolved.

LOW **L-04** RESOLVED

Anyone can freeze the empty BurnMint escrow once the policy blocks oft_store, halting the OFT

CATEGORY	DoS
LOCATION	apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/gate.rs:can_freeze(), apps/project-types/console-oft-app/contracts/solana/docs/usage.md
REVIEW	Main Review

DESCRIPTION

The Token ACL gate approves a permissionless freeze of any token account that is empty and whose owner the pubkey policy blocks. It never checks whether that owner is a protocol account. `oft_escrow` is owned by `oft_store`, and on a BurnMint OFT it is empty at rest: `send` burns what it takes in and sweeps the fee, and `lz_receive` transfers out what it mints. So as soon as the policy blocks `oft_store`, any signer can call Token ACL `FreezePermissionless` on the escrow and the gate approves it. V1 had no permissionless freeze path, so no public actor could freeze the escrow.

```rust
pub fn can_freeze(ctx: &Context<Self>) -> Result<()> {
require!(ctx.accounts.subject_blocked()?, PolicyError::NotBlocked);
ctx.accounts.require_empty()
}
```

A frozen escrow halts the OFT in both directions: the transfer into it on `send` and the mint into it on `lz_receive` fail with `AccountFrozen`. `can_thaw` refuses while `oft_store` is blocked, and after a Thawer reopens the escrow anyone can freeze it again. A normal admin action reaches this state: switching the policy to Whitelist mode before `oft_store` has a whitelist record, since an unset record reads as not whitelisted. `docs/usage.md` lists whitelisting `oft_store` as a standing duty but does not order it before the mode switch, and parity D4 describes the permissionless layer as only moving accounts "worth nothing to hold". Nothing is lost: payloads stay retryable, and unblocking `oft_store` lets anyone thaw the escrow. A LockUnlock escrow is exposed the same way while it holds nothing.

RECOMMENDATION

L-04 Anyone can freeze the empty BurnMint escrow once the policy blocks oft_store, halting the OFT

CONTINUED

Exempt protocol accounts from the permissionless freeze: let the token-manager DefaultAdmin register owners such as `oft_store`, the fee deposit's owner and the redirect escrow's owner, and have `can_freeze` refuse them, while role-gated freezes stay available. Alternatively, document in `docs/usage.md` that these owners must be whitelisted before the policy switches to Whitelist mode and must never be blocked in any mode, since `can_freeze` approves a freeze of any empty account whose owner the policy blocks.

RESOLUTION

Status: Resolved.

LOW L-05

RESOLVED

lz_receive_types_info reads a synced default ACL gate as "no config", so every receive fails to plan while a policy is selected

CATEGORY	Logical Error
LOCATION	apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive_types_info.rs:59-62, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/sync_acl_gating_program.rs:26-33, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive_types_v3.rs:153-160,212, apps/token-app/contracts/solana/token-manager/interface/src/acl/manifest.rs:123-129
REVIEW	Main Review

DESCRIPTION

Receive discovery and the receive planner disagree on when the gate's thaw manifest is required. The planner requires the full ACL bundle whenever a policy is selected and the mint's freeze authority is its Token ACL `MintConfig`, regardless of which gate the config names (`requires_acl`, `manifest.rs:128`). Discovery only names the manifest when the cached gate is not `Pubkey::default()`. That cache is written by `sync_acl_gating_program`, which stores `Pubkey::default()` both when the mint has no config and when the config's gate is `Pubkey::default()`, without telling the two apart (`sync_acl_gating_program.rs:26-32`):

```rust
// requires_acl: the gate is not a factor
policy_selected && freeze_authority == COption::Some(mint_config_address(mint))

// lz_receive_types_info: the gate is a factor
acl_thaw_metas: store
.pubkey_policy
.filter(|_| store.acl_gating_program != Pubkey::default())
.map(|_| acl::thaw_metas_address(&store.token_mint, &store.acl_gating_program)),
```

The colliding state is reachable because the field comes from Token ACL, which `console_oft` neither controls nor validates: at the pinned revision (token-manager README:7), `CreateConfig` and `SetGatingProgram` store any gating program taken from instruction data ([create_config.rs#L34-L35](https://github.com/solana-foundation/token-acl/blob/bfbaf589662631ca84f3462514620a499405c6d7/program/src/instructions/create_config.rs#L34-L35), [set_gating_program.rs#L18-L28](https://github.com/solana-foundation/token-acl/blob/bfbaf589662631ca84f3462514620a499405c6d7/program/src/instructions/set_gating_program.rs#L18-L28)), and its CLI writes `Pubkey::default()` when no gate is given

L-05 lz_receive_types_info reads a synced default ACL gate as "no config", so every receive fails to plan while a policy is selected — CONTINUED

([main.rs#L61](https://github.com/solana-foundation/token-acl/blob/bfbaf589662631ca84f3462514620a499405c6d7/clients/cli/src/main.rs#L61)). For such a mint with a selected policy, discovery omits `acl_thaw_metas`, and the planner reads the gate from the config and fails at `acl_thaw_metas.ok_or(OFTError::InvalidAclThawMetas)` (`lz_receive_types_v3.rs:212`). Every inbound message fails planning, including plain deliveries to already thawed recipients. Re-running `sync_acl_gating_program` does not help because the cache already holds the true value, although `usage.md:35-38` presents the sync as the fix for `InvalidAclThawMetas`. Naming the default gate's canonical (unallocated) manifest is enough for planning to succeed, and the execution helpers already treat that manifest as "no extras" (`manifest.rs:42-44,59-61`). Inbound traffic stays stuck, with payloads retryable, until an admin points the config at a real gate or clears the OFT's policy, which turns recipient screening off.

RECOMMENDATION

In `lz_receive_types_info`, name the canonical thaw manifest whenever a policy is selected, deriving it from the cached gate even when that gate is `Pubkey::default()`, or store the cache as `Option<Pubkey>` so "no config" and "config with a default gate" stay distinct.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01** RESOLVED

Redirect escrow requirements omit memo-required and confidential-only accounts

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/set_credit_redirect_config.rs:SetCreditRedirectConfig apps/project-types/console-oft-app/contracts/solana/docs/usage.md
REVIEW	Main Review

DESCRIPTION

`set_credit_redirect_config` validates the redirect escrow only against the frozen state, and `docs/usage.md` lists only "unfrozen when configured" and "keep the redirect escrow open" as escrow requirements. Token-2022 `process_transfer` also refuses a destination with MemoTransfer required, which always fails here because the program never issues a memo. On mints with the ConfidentialTransferMint extension it also refuses a destination whose non-confidential credits are disabled. The escrow's owner can turn on either setting at any time.

```rust
#[account(
token::mint = token_mint,
constraint = !redirect_escrow.is_frozen()
@ OFTError::RedirectEscrowFrozen,
constraint = redirect_escrow.owner != oft_store.key()
@ OFTError::InvalidRedirectEscrow,
)]
pub redirect_escrow: Option<InterfaceAccount<'info, TokenAccount>>,
```

`lz_receive` sends every credit for a blocked or frozen recipient to the configured escrow (`credit_to` in `LzReceive::apply`). With either setting on the escrow, that transfer fails with `NoMemo` or `NonConfidentialTransfersDisabled`, and every redirected receive reverts. Payloads stay retryable until the setting is turned off or the escrow is re-pointed. The docs cover the memo case only for the recipient and the fee deposit, and do not mention the confidential case for any account.

RECOMMENDATION

I-01 Redirect escrow requirements omit memo-required and confidential-only accounts CONTINUED

Add to the standing duties in `docs/usage.md` that the redirect escrow must not require incoming memos or disable non-confidential credits. Reject such an escrow in `set_credit_redirect_config` next to the existing frozen check.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02 RESOLVED

A zero-address recipient is reported as fully credited on Solana but as 0 on EVM

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/helpers/credit_redirect.rs:delivered_amount() apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive.rs:apply() apps/project-types/console-oft-app/contracts/solana/docs/solana-evm-parity-analysis.md
REVIEW	Main Review

DESCRIPTION

The EVM `_credit` gained credit redirect in this release and now reports 0 whenever the credit does not land at `_to`. A zero `_to` is minted to `0xdead`, so it now reports 0 in both `OFTReceived` and the compose message:

```solidity
address creditTo = _redirectCredit(_to, _amountLD);
if (creditTo == address(0)) creditTo = address(0xdead);
_mint(creditTo, _amountLD);
return _to == creditTo ? _amountLD : 0;
```

At `c0197e4` the zero recipient was remapped before the mint and the full amount was returned, so both chains reported the same thing.

On Solana, `bytes32(0)` decodes to `Pubkey::default()`, the System Program ID. `lz_receive` creates that key's ATA and credits it, and `delivered_amount` only zeroes redirected credits:

```rust
pub fn delivered_amount(redirected: bool, amount_ld: u64) -> u64 {
if redirected {
0
} else {
amount_ld
}
}
```

The same message therefore reports the full amount on Solana and 0 on EVM. In both cases the tokens are unrecoverable, since nobody can sign for `0xdead` or the System Program. The parity doc lists return-zero-on-redirect but not this case, so tooling that treats the

I-02 A zero-address recipient is reported as fully credited on Solana but as 0 on EVM — CONTINUED

reported amount as proof of payment reads the same message differently depending on the destination chain.

RECOMMENDATION

Record the difference in `solana-evm-parity-analysis.md`. On Solana the all-zero key is a real address rather than a sentinel, so documenting it is preferable to special-casing it in `lz_receive` and the planner.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03 RESOLVED

Conditional native-authority signer-meta requirements are not documented

CATEGORY	Documentation
LOCATION	apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/root/init_token_manager.rs:29-31, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/root/rbac_default_admin.rs:31-40, apps/token-app/contracts/solana/pubkey-policy-hook/programs/pubkey_policy_hook/src/instructions/init_transfer_hook.rs:15-29
REVIEW	Main Review

DESCRIPTION

The `init_token_manager`, `set_pending_token_manager_admin`, and `init_transfer_hook` instructions support plain signers, CPI-signing PDAs, and native multisigs through an unchecked authority account. Consequently, generated clients mark the authority as a non-signer even though plain signers and CPI-signing PDAs require `is_signer = true`; only native multisig authority accounts should remain non-signers and provide their member signers as remaining accounts.

Internal test helpers already adjust these account metas manually, but this requirement is not explained in the public integration documentation. Callers relying directly on generated instructions may therefore encounter unexpected `InvalidAuthority` or `Unauthorized` failures.

RECOMMENDATION

Document the signer-meta requirements for authority types

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-05** RESOLVED

The parity doc's advice to hold an empty account by blacklisting has no effect in Open mode

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/docs/solana-evm-parity-analysis.md:85-102, contracts/common/utils/solana/pubkey-policy/interface/src/state.rs:13-19, contracts/common/utils/solana/pubkey-policy/interface/src/pubkey_policy.rs:74-83, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/gate.rs:53-56, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/lz_receive.rs:184
REVIEW	Main Review

DESCRIPTION

Parity section D4 covers Open mode on a Token ACL mint with permissionless thaw enabled. It correctly states that Open mode never consults membership and "blocks nobody" (lines 85 and 90-91). It then ends by telling operators that if a hold on an empty account has to stick, "the lever is blacklisting the subject", because that "makes permissionless thaw refuse" (lines 100-102). The code works as the earlier lines say. `ValidatedPubkeyPolicy::read` loads and validates the subject's record, but in Open mode the result ignores its membership:

```rust
pub fn allows(self, membership: Membership) -> bool {
match self {
Self::Open => true,
Self::Blacklist => !membership.blacklisted,
Self::Whitelist => membership.whitelisted,
}
}
```

A subject with `blacklisted = true` therefore stays allowed while the policy is in Open mode. The gate's `can_thaw` approves a public thaw of its empty account, and `lz_receive` computes `policy_blocked` as false, thaws the frozen recipient account and credits it in full. An operator who follows the closing advice sets the flag and expects the hold to stick, but anyone can still thaw the account. The flag only takes effect once the policy is in Blacklist mode. Switching to Blacklist mode does not block anyone else, because a subject without a record stays allowed there.

I-05 The parity doc's advice to hold an empty account by blacklisting has no effect in Open mode — CONTINUED

RECOMMENDATION

Rewrite the end of D4 to say that a hold on an empty account needs two steps: switch the policy to Blacklist mode and set the subject's `blacklisted` flag. Note that in Blacklist mode a subject without a record stays allowed, so the switch affects only subjects that are explicitly blacklisted.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-06 RESOLVED

The successor-OFT guidance tells operators to move LockUnlock collateral, but Console OFT has no instruction or procedure for it

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/docs/usage.md:91-92, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/lib.rs:41-262, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/permanent_delegate/pd_recover.rs:16-68, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/freeze_token_account.rs:11
REVIEW	Main Review

DESCRIPTION

In a LockUnlock OFT, `oft_escrow` holds the collateral that backs tokens on remote chains. The escrow PDA is derived from `oft_store`, so a successor OFT gets a new, empty escrow. The new `usage.md` lists moving the collateral as one migration step, with no instruction or procedure for it:

```
``markdown
- **Successor OFT.** Pause, deliver or skip pending payloads to the old receiver, move LockUnlock
collateral, restore rate-limit usage with set_rate_limit_state, then switch peers.
``
```

Console OFT has no instruction that withdraws, sweeps or migrates the escrow balance (`lib.rs:41-262`), and it never closes the escrow, changes its authority or revokes its delegate. For LockUnlock, funds leave the escrow only through the credit in `lz_receive` and the fee transfer in `send`, and neither lets an admin choose the destination; the escrow burns in `send` only run for BurnMint (`send.rs:271`). The one direct path is in token_manager. A Freezer freezes the escrow through Token ACL (`freeze_token_account.rs:11`). Then a Recoverer calls `pd_recover`, which thaws the escrow, moves its balance using the Permanent Delegate signer, and freezes it again (`pd_recover.rs:16-68`). That path only exists on a Token-2022 mint whose Permanent Delegate and Token ACL authorities are held by the TokenManager root. On legacy SPL Token mints and on Token-2022 mints without a Permanent Delegate, the only remaining routes are workarounds: releasing the balance through `lz_receive` with a crafted message from a peer the admin controls, or upgrading the shared program, which affects every OFT instance. An operator who follows the guidance and switches peers leaves the successor without collateral, so its LockUnlock receives fail until it is funded, while the old balance stays locked in the old escrow.

I-06 The successor-OFT guidance tells operators to move LockUnlock collateral, but Console OFT has no instruction or procedure for it

CONTINUED

RECOMMENDATION

Replace the step in `usage.md` with a complete procedure. Name `pd_recover` and its prerequisites: a Token-2022 mint, Permanent Delegate and Token ACL authorities held by the root, a frozen source, and the Freezer and Recoverer roles. For every other mint, state that operators must either unwind the old escrow through inbound deliveries before switching peers or fund the successor escrow separately, and that the old escrow balance has no administrative withdrawal path.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-07** RESOLVED

pd_recover credits the destination net of Token-2022 transfer fees while FundsRecovered reports the gross amount

CATEGORY	Documentation
LOCATION	`packages/mirror-layerzero-labs-audit-external/apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/permanent_delegate/pd_recover.rs:PdRecover::apply:L82-L110`
REVIEW	Main Review

DESCRIPTION

PdRecover::apply passes the requested gross amount directly to Token-2022 transfer_checked without checking TransferFeeConfig or reconciling the destination delta. The instruction is documented as the Solana equivalent of EVM recoverFunds, whose post-condition transfers exactly the requested amount to the destination. On a fee-bearing Token-2022 mint, source funds decrease by the gross request while the destination receives only the net amount and the withheld fee remains separately withdrawable; FundsRecovered still emits the gross request.

Impact: For a requested recovery of 10,000 with a 1% fee, the source is reduced by 10,000, destination balance increases by only 9,900, and 100 is withheld and can be swept by the configured withdraw-withheld authority, while FundsRecovered.amount reports 10,000. Thus documented recovery semantics and off-chain event accounting overstate the destination recovery. Impact is bounded by the trusted mint fee configuration.

RECOMMENDATION

Document in the token_manager README `pd_recover` section and in the D9 entry of the parity analysis that, on a mint with a live Token-2022 transfer fee, the destination receives the recovered amount minus the fee.

PROOF OF CONCEPT `https://github.com/GuardianOrg/SentryRepo-LayerZero-Solana-OApp---Console-V2--Round-1-01a0f2cb582775b489aa314677d6b27d/blob/refinery/poc-01a0f303-6582-76f0-a9b0-789bad783876/test/refinery-poc/pd-recover-ignores-token-2022-transfer-fees-silently-shortin-6f4a4f5e.rs`

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08 RESOLVED

The README names the wrong error for init_acl rejecting another mint's policy

CATEGORY	Documentation
LOCATION	apps/token-app/contracts/solana/token-manager/README.md:196-199, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/init_acl.rs:68-70, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/errors.rs:3-4,13-14
REVIEW	Main Review

DESCRIPTION

Step 5 of the Token ACL setup in the token-manager README says `init_acl` rejects "a policy for another mint (`InvalidConfig`)" (lines 196-199). The handler raises a different error. After decoding the policy it compares the policy's mint with the root's mint and fails with `WrongPolicyMint`:

```rust
let policy =
PubkeyPolicy::try_deserialize(&mut &ctx.accounts.pubkey_policy.try_borrow_data()?[..])?;
require_keys_eq!(policy.mint, ctx.accounts.mint.key(), TokenManagerError::WrongPolicyMint);
```

In `TokenManagerError` these are two separate codes with separate meanings. `InvalidConfig` (7000) is "Invalid canonical Token ACL configuration", which the ACL helpers raise for a bad MintConfig. `WrongPolicyMint` (7005) is "The bound pubkey policy is for another mint". An integrator or operator tool that follows the README and matches on `InvalidConfig` to detect a wrong-mint policy never sees that code. A real `WrongPolicyMint` failure then goes unrecognized, or a true `InvalidConfig` gets misread as a policy-binding mistake. The README's own hook section (line 270) and the gate both use `WrongPolicyMint` for the same condition, so the setup step is the one inconsistent statement.

RECOMMENDATION

Change the README's step 5 to name `WrongPolicyMint` as the error `init_acl` raises for a policy bound to another mint, matching `init_acl.rs:70` and the hook section.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09 RESOLVED

The README's pause-authority setup is one pd_recover cannot resume

CATEGORY	Documentation
LOCATION	apps/token-app/contracts/solana/token-manager/README.md:84-93, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/permanent_delegate/pd_recover.rs:78-80,99-101,123-135
REVIEW	Main Review

DESCRIPTION

The token-manager README says pausing and resuming "stay with the native pause authority (e.g. a 1-of-n multisig), which signs Token-2022 directly", and that a root holds `Pause` only so `pd_recover` can move funds while the mint is paused. Token-2022 stores a single pause authority, and the README's table assigns it to the root's Pause signer, so the two sentences describe mutually exclusive setups. The only configuration satisfying both is a native multisig with the Pause signer as a member, which `pd_recover` does not support because it resumes and re-pauses with the Pause signer itself as the authority and passes no multisig signers:

```rust
let ix =
toggle(&self.token_program.key(), &self.mint.key(), &self.pause_signer.key(), &[])?;
invoke_as(
&self.token_manager.key(),
&[(SignerKind::Pause, bump)],
&ix,
&[info, self.pause_signer.to_account_info(), self.token_program.to_account_info()],
)
```

Token-2022's `process_toggle_pause` requires the supplied authority to equal the stored one, so under that setup every `pd_recover` on a paused mint fails with `OwnerMismatch`. Recovery while paused then needs the pause multisig to co-sign a transaction that resumes, calls `pd_recover` and pauses again, so the instruction's own resume and re-pause logic goes unused. The only setup in which that logic works is the Pause signer holding the authority directly, and there the native key can no longer pause or resume without DefaultAdmin first moving the authority out with `set_native_authority`. The README describes a setup that the instruction's documented paused-mint handling does not support.

I-09 The README's pause-authority setup is one pd_recover cannot resume CONTINUED

RECOMMENDATION

State in the README that `pd_recover` resumes a paused mint only when the root's Pause signer is itself the pause authority, and that a native multisig authority must instead bundle resume, `pd_recover` and pause in one transaction. Alternatively, accept a native multisig pause authority in `PdRecover::set_paused` by passing the stored authority with the Pause signer as its signing member.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-10 RESOLVED

The README says a zero-fee send can omit the fee deposit, but decimal dust still requires it

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/README.md:128-135, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/send.rs:217-243,366-376,424-435
REVIEW	Main Review

DESCRIPTION

The Console OFT README defines the fee as the configured bps and says the fee hook group is "present only when a fee is charged" and that "a zero-fee send never uses the deposit and can omit its account", even while the deposit is frozen. The amount `send` sweeps to the deposit is not the bps fee but everything not bridged. `debit_view` keeps the full amount as sent and removes dust only from the received amount, so `fee_ld` includes the shared-decimal dust:

```rust
let oft_fee = fee_config::get_oft_fee(oft_store.default_fee_bps, fee_config, amount_ld);
let amount_received_ld = oft_store.remove_dust(amount_ld - oft_fee);
...
let fee_ld = amount_sent_ld - amount_received_ld;
...
let endpoint_accounts = if fee_ld == 0 {
rest
} else {
let (fee_hooks, rest) = TransferHookAccounts::take_prefix(&hook, rest)?;
Self::collect_fee(ctx, fee_ld, fee_hooks.cpi_accounts())?;
rest
};
```

On a mint whose local decimals exceed the shared decimals, a 0-bps send of any amount that is not a multiple of `ld2sd_rate` has a non-zero `fee_ld`, so `send` expects a fee hook group and transfers the dust to the deposit. An integrator that follows the README and omits the deposit on a 0-bps route gets `MissingFeeDeposit`, one that omits the fee hook group on a hook mint has the Endpoint accounts parsed as that group, and while the deposit is frozen every such send reverts at the native transfer, although the README tells operators

I-10 The README says a zero-fee send can omit the fee deposit, but decimal dust still requires it — CONTINUED

those routes keep working. The `fee_deposit` account comment ("Optional when the collected fee (including decimal dust) is zero") states the actual rule, but the README contradicts it.

RECOMMENDATION

Change both README sentences to say that the fee deposit and the fee hook group are required whenever the collected amount, the bps fee plus decimal dust, is non-zero, which includes 0-bps sends of amounts that are not a multiple of the shared-decimal rate. State that a frozen deposit therefore blocks those sends as well as fee-bearing ones.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-11 RESOLVED

The successor TokenManager migration has the old root hand over the authority step 1 already moved to a signing key

CATEGORY Documentation

LOCATION apps/token-app/contracts/solana/token-manager/docs/usage.md:47-58, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/native_authority/set_native_authority.rs:24-26,83-90,101, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/manage_acl.rs:71-87

REVIEW Main Review

DESCRIPTION

The procedure in `docs/usage.md` for migrating a mint from one TokenManager program to a successor program says that if every authority sits on the old root, step 1 first moves one of them to a signing key with `set_native_authority`, so that key can create the successor root. Step 3 then has the old root run `set_acl_gating_program` and `set_native_authority` for Acl, PermissionedBurn, PermanentDelegate and Pause to the successor's signer PDAs. The old root signs every handover only as its own signer PDA for that kind, so it can no longer move the authority step 1 moved away:

```rust
#[account(seeds = [SIGNER_SEED, token_manager.key().as_ref(), &[kind as u8]], bump)]
pub signer: UncheckedAccount<'info>,
invoke_as(&state, &[(kind, ctx.bumps.signer)], &ix, &infos)?;
```

Followed as written, the procedure therefore stops at step 3 whichever of Acl, PermissionedBurn or PermanentDelegate step 1 moved. If it moved PermissionedBurn or PermanentDelegate, step 3's `set_native_authority` for that kind reverts because Token-2022 no longer sees the old PDA as the authority. If it moved Acl, the signing key is now Token ACL's logical admin, so the first call of step 3, `set_acl_gating_program` (which signs as the old Acl PDA), and the later `set_native_authority(Acl)` both fail with Token ACL `InvalidAuthority`, and the successor gate is never wired in by the documented calls. The procedure only completes once the operator works out that the signing key must hand its authority over natively, which the document never says.

RECOMMENDATION

I-11 The successor TokenManager migration has the old root hand over the authority step 1 already moved to a signing key CONTINUED

In step 3, state that the signing key itself hands over the authority moved in step 1 (Token-2022 `SetAuthority`, or Token ACL `SetGatingProgram` and `SetAuthority` for Acl), while the old root moves only the authorities it still holds.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01** RESOLVED

set_acl_permissions reverts on unchanged flags "as on EVM", but EVM's two-flag setter stores and emits

CATEGORY	Documentation
LOCATION	apps/token-app/contracts/solana/token-manager/README.md:195-197, apps/token-app/contracts/solana/token-manager/programs/token_manager/src/instructions/acl/manage_acl.rs:41-50, apps/project-types/console-oft-app/contracts/solana/programs/console_oft/src/instructions/rate_limiter/set_rate_limit_global_config.rs:18-27, contracts/common/utils/evm/upgradeable/contracts/rate-limiter/RateLimiterBaseUpgradeable.sol:531-536
REVIEW	Remediation Review 1

DESCRIPTION

`set_acl_permissions` now fails with `AclPermissionsIdempotent` (7012) when Token ACL's permissionless freeze and thaw flags already hold the requested values. Both the handler's doc comment and the README justify this as EVM behaviour ("as EVM's boolean setters do", "As on EVM, setters of a binary state (`pause`, `unpause`, `set_acl_permissions`) fail"). Token ACL permissionless freeze and thaw have no EVM counterpart, and the one EVM setter with the same shape, two booleans written together, does not revert on an unchanged value:

```solidity
function _setRateLimitGlobalConfig(RateLimitGlobalConfig memory _globalConfig) internal virtual {
RateLimiterStorage storage $ = _getRateLimiterStorage();
$.useGlobalStateFlag = _globalConfig.useGlobalState;
$.isGloballyDisabledFlag = _globalConfig.isGloballyDisabled;
emit RateLimitGlobalConfigUpdated(_globalConfig);
}
```

Console OFT's `set_rate_limit_global_config` mirrors that EVM setter and stores and emits on every call, so the deployment now has two two-flag admin setters with opposite behaviour on an unchanged request. Before this change both stored and emitted. An operator or tooling author who reads "binary setters revert, as on EVM" expects a repeated `set_rate_limit_global_config` to fail, or a repeated `set_acl_permissions` to succeed and emit, as it did previously and as EVM's two-flag setter does. Either expectation is wrong for one of the two instructions.

RECOMMENDATION

I-01 set_acl_permissions reverts on unchanged flags "as on EVM", but EVM's two-flag setter stores and emits — CONTINUED

Remove the EVM attribution for `set_acl_permissions` from the README and the `ManageAcl::permissions` doc comment, describing the revert as a Solana-only rule that follows the program's binary-state convention. Document `set_rate_limit_global_config` as a deliberate value setter that stores and emits on every call to match EVM `setRateLimitGlobalConfig`, so the exception is explicit.

— RESOLUTION

Status: Resolved.

MEDIUM M-01 RESOLVED

OApp macro rejects module inner attributes.

CATEGORY Logical Error LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L86

REVIEW OApp · Main Review 1

DESCRIPTION

The anchor-trait Assembly::assemble() in contracts/common/framework/solana/anchor-trait/src/assembly.rs re-emits the user-authored module's attributes (ItemMod.attrs) without distinguishing outer style (#[...]) from inner style (#![...]). Both are dumped uniformly via #(#mod_attrs)* at a position outside the generated program module

RUST

```rust
quote! {
#prelude
#wrapper_defs
#default_impls
#extend_macro

#(#mod_attrs)*          // ⟵ inner attrs (#![...]) land here, outside the mod
#extra_mod_attrs
#[program]
#mod_vis mod #mod_ident {
use super::*;
#filtered_items
#entrypoints
}
}
```

quote! preserves AttrStyle on round-trip, so a developer-written #![X] is re-emitted as #![X] in the new position. Because #prelude, #wrapper_defs, #default_impls, and #extend_macro always emit tokens before this point, the inner attribute is never in a legal position (first tokens of an enclosing item). rustc rejects it with a hard compile error:

TEXT

```
error: an inner attribute is not permitted in this context
--> src/lib.rs
|
175 | #[oapp::oapp(state = CounterState, role_type = CounterRoleType)]
| ------------------------------------------------------------------ the inner attribute doesn't annotate this module
176 | pub mod counter {
177 |     #![deny(clippy::arithmetic_side_effects)]
|     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
|
= note: inner attributes, like `#![no_std]`, annotate the item enclosing them
```

This blocks developers from using idiomatic module-scoped Rust smart-contract hardening pragmas inside the #[oapp] module.

MEDIUM M-01 CONTINUED 2/2 RESOLVED

OApp macro rejects module inner attributes.

RECOMMENDATION

Consider partitioning mod_attrs by syn::AttrStyle and route each style to the position where rustc accepts it: outer attrs stay outside the generated module (they annotate the module from outside); inner attrs move to the first tokens inside the generated module body (they annotate the module from inside). Both land on mod #mod_ident semantically - the! is a positional marker, not a functional distinction.

RUST

```rust
// contracts/common/framework/solana/anchor-trait/src/assembly.rs
let (inner_attrs, outer_attrs): (Vec<_>, Vec<_>) = mod_attrs
.into_iter()
.partition(|a| matches!(a.style, syn::AttrStyle::Inner(_)));

quote! {
#prelude
#wrapper_defs
#default_impls
#extend_macro

#(#outer_attrs)*
#extra_mod_attrs
#[program]
#mod_vis mod #mod_ident {
#(#inner_attrs)*
use super::*;
#filtered_items
#entrypoints
}
}
```

RESOLUTION

LayerZero Team - Resolved.

LOW L-01 RESOLVED

Payer-funder mismatch in enforced options.

CATEGORY Logical Error LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/set_enforced_options.rs#L51

REVIEW OApp · Main Review 1

DESCRIPTION

The payer in StdSetEnforcedOptions (set_enforced_options.rs:44) is an unconstrained Signer<'info>, independent of the account that originally funded the EnforcedOptions PDA in init_enforced_options. Both instructions declare payer as a separate field from authority, with no constraint tying them together or tracking who originally funded the PDA. When the options buffer shrinks during a set_enforced_options call, Anchor's realloc::payer = payer directive routes the reclaimed rent lamports to the current transaction's payer - not the account that originally paid the rent in init_enforced_options. This creates a funder/refund-recipient mismatch: account A funds the PDA at initialization, but account B receives the refund when the PDA shrinks.

RECOMMENDATION

Consider separating the payer (who funds growth) from a refunder (who receives lamports on shrink) and handling the refunds manually. Alternatively, consider documenting this behavior explicitly so integrators are aware that rent refunds go to the current payer, not the original funder.

RESOLUTION

LayerZero Team - Resolved.

LOW L-02 RESOLVED

Admin transfer reverts if OApp not registered.

CATEGORY Validation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/accept_default_admin_transfer.rs#L25-L33

REVIEW OApp · Main Review 1

DESCRIPTION

The #[oapp] macro unconditionally injects a set_delegate CPI call into the accept_default_admin_transfer handler (accept_default_admin_transfer.rs). This CPI targets the endpoint's SetDelegate instruction, which requires the OAppRegistry PDA (seeds [OAPP_SEED, oapp.key]) to exist and be initialized. The OAppRegistry is only created when the developer calls endpoint_cpi::register_oapp during their custom initialize instruction; a step the #[oapp] macro neither generates nor enforces. If a developer deploys an OApp using the #[oapp] macro but omits the register_oapp CPI in their initialization handler, the RBAC system works normally; setup_default_admin! sets the admin, roles can be granted, and begin_default_admin_transfer succeeds since it only modifies program state. However, when the pending admin calls accept_default_admin_transfer, the RBAC portion (StdAcceptDefaultAdminTransfer::apply) completes but the subsequent set_delegate CPI fails because the OAppRegistry PDA does not exist. An OApp deployed without calling register_oapp during initialization has a permanently bricked admin transfer flow.

RECOMMENDATION

Consider adding a guard in the injected accept_default_admin_transfer override that checks whether the OAppRegistry PDA exists before attempting the set_delegate CPI. If the registry does not exist, either skip the CPI (with a warning event) or return a descriptive error. Alternatively, document that register_oapp is a mandatory step during initialization.

RESOLUTION

LayerZero Team - Resolved.

LOW L-04 RESOLVED

Missing #[repr(u8)] enable role seed collision.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/role_type.rs#L10-L32

REVIEW OApp · Main Review 1

DESCRIPTION

The role_type_derive_impl function in rbac/macros/src/role_type.rs validates only that the enum is non-empty and all variants are unit variants. It does not validate that the enum carries #[repr(u8)]. The RoleType::seed() method in rbac/src/traits.rs converts the role to u8 via Into<u8> and uses that single byte as the PDA seed component: [ROLE_MEMBER_SEED, state.key(), seed_byte, member.key()]. The #[only_role] constraint validates authorization purely by checking PDA existence at these seeds. Without #[repr(u8)] enforcement, two collision paths exist: 1. No #[repr] + manual Into<u8> collision: A developer omits #[repr(u8)] and writes a manual Into<u8> impl that maps two distinct roles to the same byte value. 1. #[repr(u16)] + truncation: A developer uses #[repr(u16)] with a variant value like Special = 256, then implements Into<u8> with self as u8; truncating 256 to 0, which collides with DefaultAdmin.

RECOMMENDATION

Consider adding validation in role_type_derive_impl to verify the enum carries #[repr(u8)] using input.attrs inspection. Emit a compile-time error if the attribute is missing.

RESOLUTION

LayerZero Team - Resolved.

LOW L-05 RESOLVED

only_role Does Not Enforce Signer.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/only_role.rs#L34-L44

REVIEW OApp · Main Review 1

DESCRIPTION

The #[only_role] macro currently enforces role membership by checking for a valid RoleMember PDA derived from (state, role, authority.key()), but it does not enforce that authority is a signer. This proves that the provided authority public key has the role, but it does not by itself prove that the caller controls that key (i.e., that it signed the transaction). This means the authorization guarantee depends on integrators correctly declaring authority as Signer<'info> (or adding an equivalent signer constraint). If they instead use a non-signer account type, the role check can be satisfied using a user-supplied pubkey. In that case, an attacker can pass a privileged account's public key and matching role PDA without possessing its private key, resulting in unauthorized execution of role-protected instructions in downstream programs. While all authority fields in this repository's OApp module are currently typed as Signer<'info>, from an RBAC framework perspective #[only_role] creates a security gap for external integrators because it does not enforce signer status on its own.

RECOMMENDATION

Update #[only_role] to enforce signer proof by default (for example via a generated authority.to_account_info().is_signer constraint, or a compile-time requirement that authority is Signer<'info>).

RESOLUTION

LayerZero Team - Resolved.

LOW L-06 RESOLVED

Fully-qualified attribute paths not matched.

CATEGORY Validation

LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L78, contracts/common/framework/solana/anchor-trait/src/overrides.rs#L179

REVIEW OApp · Main Review 1

DESCRIPTION

Two path predicates in anchor-trait use syn::Path::is_ident, which is strict-1-segment. Qualified forms of the same attribute slip through. 1. #[program] dedup (assembly.rs)

RUST

```rust
let mod_attrs: Vec<_> =
mod_attrs.iter().filter(|a| !a.path().is_ident("program")).collect();
```

#[anchor_lang::prelude::program] or #[::anchor_lang::prelude::program] is not filtered, so the generated module carries two #[program] attributes and Anchor's program macro runs twice. 1. #[<domain>_instruction] override recognition (overrides.rs)

RUST

```rust
/// Returns `true` if `attr`'s last path segment matches the configured attribute name.
fn is_override_attr(attr: &Attribute, cfg: OverrideConfig) → bool {
attr.path().is_ident(cfg.attr_name)
}
```

Doc says last-segment; code uses is_ident. #[crate::oapp_instruction] bypasses recognition - if the handler name doesn't collide with a managed instruction, it passes through as a regular item and the framework silently emits the default.

RECOMMENDATION

Consider an exact-match allowlist of Anchor's canonical paths for #[program]

RUST

```rust
#[program]
#[anchor_lang::program]
#[::anchor_lang::program]
#[anchor_lang::prelude::program]
#[::anchor_lang::prelude::program]
```

For the override marker recognition in is_override_attr, compare only the final path segment:

RUST

```rust
// overrides.rs
attr.path().segments.last().is_some_and(|seg| seg.ident == cfg.attr_name)
```

RESOLUTION

LayerZero Team - Resolved.

LOW L-07 RESOLVED

Override Handler Cannot Access Nested Module Helpers.

CATEGORY Logical Error LOCATION contracts/common/framework/solana/anchor-trait/src/overrides.rs#L119-L133

REVIEW OApp · Main Review 1

DESCRIPTION

anchor-trait relocates override handlers into generated sibling wrapper modules, which changes their lexical scope relative to the user-authored #[oapp] / # [rbac] module. As a result, an overridden instruction such as lz_receive cannot reliably access a nested helper module defined inside the same macro-managed module. For example:

TEXT

```
#[oapp::oapp(...)]
pub mod my_oapp {
mod utils {
pub fn helper() { ... }
}

#[oapp_instruction]
pub fn lz_receive(...) -> Result<()> {
utils::helper();
Ok(())
}
}
```

The code above cannot compile because utils is inside my_oapp, but the relocated override handler will be placed in a sibling module rather than remaining inside my_oapp, roughly like below

TEXT

```
mod __oapp_instruction__lz_receive_handler {
use super::*;
pub fn lz_receive(...) -> Result<()> {
utils::helper(); // now looked up from this sibling wrapper scope
Ok(())
}
}

#[program]
pub mod my_oapp {
use super::*;

mod utils {
pub fn helper() { ... }
}

pub fn lz_receive(...) -> Result<()> {
__oapp_instruction__lz_receive_handler::lz_receive(...)
}
}
```

For a workaround, developers who need nested helper modules for overridden handlers must either inline the logic inside the handler itself or move the helper module outside the #[oapp] module into outer file scope or another utility module that remains visible to the generated wrapper modules.

LOW L-07 CONTINUED 2/2 RESOLVED

Override Handler Cannot Access Nested Module Helpers.

RECOMMENDATION

One option is to avoid moving override handlers into isolated wrapper modules, which would preserve the user module's lexical scope and allow overridden handlers to access nested same-module helper modules naturally. Alternatively, document this behavior clearly and warn developers that helper modules used by overridden handlers should be placed outside the macro-managed module, such as in outer scope or a separate utility module.

RESOLUTION

LayerZero Team - Resolved.

LOW L-08 ACKNOWLEDGED

Peer/options accounts cannot be closed to reclaim rent.

CATEGORY Best Practices LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/set_peer.rs#L44-L50

REVIEW OApp · Main Review 1

DESCRIPTION

The set_peer() instruction generated by the #[oapp] macro creates an OAppPeer PDA per (oapp, eid) pair via init_if_needed, paying rent from the payer signer. The generated StdSetPeer accounts struct has no close =... directive, and no separate close_peer() / remove_peer() / disable_peer() instruction is emitted by the macro. The OAppPeer struct itself contains only address: [u8; 32] and bump: u8 - there is no enabled/disabled flag.

As a result, once an OApp admin enables a remote chain by calling set_peer(), the only path to "disable" that peer on Solana is to call set_peer() again with peer = [0u8; 32]. The consequences are: 1. The rent-paid PDA stays allocated forever; the SOL paid at initialization is unrecoverable. 1. The emitted PeerSet { eid, peer: [0; 32] } event is indistinguishable from "freshly initialized but not yet configured" by off-chain indexers. This is the framework-level macro used by every OApp built on top of it, so the issue compounds across chains/EIDs and across deployments. This is also true for the EnforcedOptions accounts

RECOMMENDATION

Add a close_peer() (or remove_peer()) instruction to the #[oapp] macro's default impls that Requires default_admin_role (same RBAC as set_peer()). Uses Anchor's close = <rent_destination> constraint on the peer: Account<'info, OAppPeer> so that the lamports are returned to the admin / payer. Emits a distinct event (e.g. PeerClosed { eid }) so off-chain indexers can differentiate "explicitly disabled" from "set to zero address" from "never initialized". Alternatively, change SetPeerParams::peer to Option<[u8; 32]> and close the PDA when None is supplied. Apply this change to EnforcedOptions as well

RESOLUTION

LayerZero Team - Acknowledged. The current interface was intentionally retained and no close path was added.

INFO I-01 RESOLVED

EnforcedOptions space vs field order mismatch.

CATEGORY Informational LOCATION apps/oapp-app/contracts/solana/macros/src/state_types.rs#L35 REVIEW OApp · Main Review 1

DESCRIPTION

In state_types.rs, the EnforcedOptions struct is declared with field order options: Vec<u8> then bump: u8. Borsh serializes in declaration order, producing the on-disk layout: discriminator(8) + options_len(4) + options_data(N) + bump(1). However, the space() function's doc comment states Layout: discriminator (8) + bump (1) + vec_len_prefix (4) + options_data, placing bump before options; the opposite of the actual Borsh layout.

RECOMMENDATION

Consider correcting the doc comment to reflect the actual Borsh layout: /// Layout: discriminator (8) + vec_len_prefix (4) + options_data + bump (1).

RESOLUTION

LayerZero Team - Resolved.

INFO I-02 RESOLVED

Override can desync OAppInfo and SDK discovery.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/lib.rs#L56-L59 REVIEW OApp · Main Review 1

DESCRIPTION

The #[oapp_instruction] / #[rbac_instruction] override model accepts arbitrary Context<T>, but OAppInfo.instructions.* stays at the default standard version (for example 1) unless the developer explicitly marks that instruction as custom during init_oapp_info!. In oapp_info, instruction version bytes are SDK discovery signals: 0 means custom discovery (SDK falls back to ExtraAccountMetaList), while >0 means standard discovery at that version (SDK uses predefined Std* account templates). This creates a silent app-to-SDK contract mismatch: On-chain: instruction account layout has changed (custom Context<T>). Off-chain: SDK still reads version 1 and builds the predefined Std* account template. As a result, callers may see either: functional breakage (account deserialization/position mismatch and revert), or unexpected success with unintended semantics when extra accounts are ignored. The framework does not enforce or warn that custom overrides must be paired with init_oapp_info!(..., [InstructionName]) (version 0, custom discovery).

RECOMMENDATION

Consider adding an explicit consistency guard between override collection and codegen; when an instruction is overridden with non- Std* accounts, require it to be marked custom in init_oapp_info! (or fail/warn at compile time).

RESOLUTION

LayerZero Team - Resolved.

INFO I-03 RESOLVED

Misleading set_delegate mention in macro doc.

CATEGORY Informational LOCATION apps/oapp-app/contracts/solana/macros/src/lib.rs#L12 REVIEW OApp · Main Review 1

DESCRIPTION

The doc comment on the #[oapp] proc-macro in apps/oapp-app/contracts/solana/macros/src/lib.rs states

RUST

```
/// Uses the `anchor-trait` collect → generate → assemble pipeline to emit all
/// standard OApp instructions. Instructions with a framework-provided default
/// (e.g. `set_peer`, `set_delegate`) are generated automatically; instructions
/// without a default (e.g. `lz_receive`) must be supplied by the implementor
/// via `#[oapp_instruction]`.
```

set_delegate is not an auto-generated default OApp instruction. The OAppInstructionSet declared in apps/oapp-app/contracts/solana/macros/src/instructions/oapp/mod.rs contains 10 entries: SetPeer, GetPeer, InitEnforcedOptions, SetEnforcedOptions, GetEnforcedOptions, NextNonce, IsComposeMsgSender, LzReceiveTypesInfo, LzReceiveTypesV2, LzReceive; and no SetDelegate. The endpoint_cpi::set_delegate wrapper is only ever invoked from the unconditional accept_default_admin_transfer override.

RECOMMENDATION

Consider replacing set_delegate in the example list with an actual auto-generated default such as get_peer or init_enforced_options. Optionally consider adding a sentence noting that endpoint delegate rotation is intentionally bound to the accept_default_admin_transfer flow, mirroring OAppCoreRBACUpgradeable.

RESOLUTION

LayerZero Team - Resolved.

INFO I-04 RESOLVED

lz_receive_types_info accounts undocumented.

CATEGORY Best Practices

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/lz_receive_types_info.rs, apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs

REVIEW OApp · Main Review 1

DESCRIPTION

The off-chain LayerZero Executor invokes lz_receive_types_info with a fixed 2-account list, in this order: oapp_account, then the lz_receive_types_accounts PDA at [LZ_RECEIVE_TYPES_SEED, oapp_account.key()]. This is documented on the LZ docs site https://docs.layerzero.network/v2/developers/solana/oapp/overview#how-lz_receive_types_v2-works, but is not surfaced anywhere in the framework. The framework's spec at apps/oapp-app/contracts/solana/macros/src/instructions/oapp/lz_receive_types_info.rs is a required override with no compile-time check on the accounts shape, so a developer can declare a context struct of any size and the framework will compile it. Mismatch with the Executor's 2-account call is a runtime failure, discovered only after deployment. The same exists for lz_compose_types_info.,

RECOMMENDATION

Consider promoting lz_receive_types_info / lz_compose_types_info from a required override to a Provided default with the canonical 2-account context and handler. Devs can still override via #[oapp_instruction] when needed, but get the correct shape for free by default.

RESOLUTION

LayerZero Team - Resolved.

INFO I-05 RESOLVED

Unused error variant OAppError::InvalidPeer.

CATEGORY Informational LOCATION apps/oapp-app/contracts/solana/src/errors.rs#L13 REVIEW OApp · Main Review 1

DESCRIPTION

OAppError::InvalidPeer is declared at apps/oapp-app/contracts/solana/src/errors.rs but is never used anywhere in the repo. It is surfaced to clients via the IDL but cannot be emitted on-chain.

RECOMMENDATION

Consider removing the variant, or adding the intended set_peer validation that was meant to raise it.

RESOLUTION

LayerZero Team - Resolved.

INFO I-06 ACKNOWLEDGED

RBAC role-admin mapping is fixed at compile time.

CATEGORY Informational LOCATION contracts/common/utils/solana/rbac/macros/src/instructions/grant_role.rs#L32

REVIEW OApp · Main Review 1

DESCRIPTION

The Solana RBAC supports hierarchical role admins via RoleType::role_admin() (contracts/common/utils/solana/rbac/src/traits.rs), and grant_role / revoke_role use it when deriving the admin PDA

RUST

```
// contracts/common/utils/solana/rbac/macros/src/instructions/grant_role.rs
let role_admin_seed = quote!(&::rbac::traits::RoleType::seed(&#role_type::role_admin(¶ms.role)));
```

But role_admin() is resolved at macro-expansion time; the role→admin mapping is fixed in source. There is no on-chain instruction analogous to OpenZeppelin's _setRoleAdmin(role, adminRole), which EVM contracts (including AccessControlDefaultAdminRulesUpgradeable, the base of OAppCoreRBACUpgradeable) can expose to change role graphs at runtime. Changing a role's admin on Solana requires a full program upgrade. Additionally, role_admin(&self) → Self receives no context/accounts, so even overriding the trait can't read on-chain state to implement runtime mutability.

RECOMMENDATION

Either consider adding an on-chain set_role_admin instruction (full EVM parity) or documenting the divergence explicitly; role hierarchy is static and changing it requires a program upgrade.

RESOLUTION

LayerZero Team - Acknowledged. The compile-time role-admin mapping was intentionally retained.

INFO I-07 RESOLVED

Marker attribute macro bodies are unreachable.

CATEGORY Informational

LOCATION apps/oapp-app/contracts/solana/macros/src/lib.rs; contracts/common/utils/solana/rbac/macros/src/lib.rs

REVIEW OApp · Main Review 1

DESCRIPTION

The framework declares three #[proc_macro_attribute] stubs that users apply to override handlers: oapp_instruction() at apps/oapp-app/contracts/solana/macros/src/lib.rs#L29, composer_instruction() at apps/oapp-app/contracts/solana/macros/src/lib.rs#L56, and rbac_instruction() at contracts/common/utils/solana/rbac/macros/src/lib.rs#L56. Each body today is a trivial identity pass-through (item). Despite being declared as proc-macro attributes, these functions never execute in normal usage. Attribute expansion is outer-first: the enclosing #[oapp::oapp], #[oapp::composer], or #[::rbac::rbac] attribute consumes the whole module as a TokenStream and strips inner #[oapp_instruction] / #[composer_instruction] / #[rbac_instruction] markers via InstructionOverrides::wrap_handler() before Rust's attribute resolver sees them. The markers are recognised purely by string match on OverrideConfig::attr_name in anchor-trait 's overrides.rs - the stubs themselves are never invoked as macros. This is safe today because each body is identity. However, if a future maintainer adds logic to any of these bodies (e.g. validation, rewriting, diagnostics), that logic will silently not run in the normal flow. Tests applied via the enclosing domain macro will not exercise the stub.

RECOMMENDATION

Add a comment to each stub documenting that the body is unreachable under normal usage, and that logic intended to affect real expansions must be placed in the enclosing domain macro (oapp::expand(), composer::expand(), rbac::expand()) or in the shared InstructionOverrides pipeline - not in the stub.

RESOLUTION

LayerZero Team - Resolved.

INFO I-08 ACKNOWLEDGED

Duplicate use super::* in generated module.

CATEGORY Informational LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L90

REVIEW OApp · Main Review 1

DESCRIPTION

Assembly::assemble() always injects use super::*; as the first item inside the generated program module. When two domain macros are stacked - e.g. #[oapp::oapp] followed by #[::rbac::rbac] - each macro runs its own assemble() pass. The second pass re-emits the module with a fresh use super::*; while preserving the prior pass's use super::*; as part of filtered_items(). As a result, the final expanded module contains two identical use super::*; imports. Concretely, after #[oapp::oapp] expands in apps/oapp-app/contracts/solana/macros/src/oapp.rs, its output contains # [::rbac::rbac(...)] mod counter { use super::*;... }. When the RBAC attribute is then expanded and its Assembly::assemble() re-wraps the module, the preserved body (passed through InstructionOverrides::filtered_items()) still carries the original use super::*;, and a second one is prepended by the new emission. The duplication is harmless to compilation because use declarations are idempotent, but it is unnecessary noise in the expanded output (visible via cargo expand) and grows linearly with the number of stacked domain macros (e.g. #[oapp] + # [composer] + #[rbac] would produce three copies).

RECOMMENDATION

Either fix or acknowledge.

RESOLUTION

LayerZero Team - Acknowledged. Duplicate imports were accepted as harmless.

INFO I-09 RESOLVED

endpoint_cpi Helpers Can Panic on Short Slices.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L40 REVIEW OApp · Main Review 1

DESCRIPTION

Several CPI helpers in oapp::endpoint_cpi index into the caller-supplied accounts slice before verifying its length. As a result, undersupplied remaining_accounts may trigger uncontrolled aborts (e.g., ProgramFailedToComplete) instead of returning a structured error such as AccountNotEnoughKeys. While malformed input still results in transaction failure, this unclean failure mode degrades debuggability and integration ergonomics. The pattern appears to be consistent across multiple helpers, including register_oapp, set_delegate, send, clear, send_compose, and clear_compose.

RECOMMENDATION

Consider adding explicit slice-length checks before any positional accounts[...] access in all endpoint_cpi helpers, and return a standard error on undersupplied input.

RESOLUTION

LayerZero Team - Resolved.

INFO I-10 RESOLVED

Events omit the state instance pubkey.

CATEGORY Events LOCATION contracts/common/utils/solana/rbac/src/events.rs#L5-L28 REVIEW OApp · Main Review 1

DESCRIPTION

The rbac framework documents multi-instance programs as a supported feature - the RoleMember PDA is seeded on state.key() so different state instances get isolated role namespaces. However, none of the emitted events include the state.key() (nor any other instance identifier), so off-chain consumers cannot tell which instance an event refers to.

The affected event types are: RoleGranted { role, account, sender } - emitted by grant_role()

RoleRevoked { role, account, sender } - emitted by revoke_role(), renounce_role(), and accept_default_admin_transfer() (for the outgoing admin)

DefaultAdminTransferStarted { new_admin } - emitted by begin_default_admin_transfer()

In a multi-instance program, two independent grant_role() calls on two different state instances produce identical RoleGranted payloads if they happen to grant the same role to the same account. An off-chain indexer, governance dashboard, or alerting system subscribing to these events has no reliable way to attribute each event to the instance it originated from. The same concern applies symmetrically to the OApp-side events PeerSet and EnforcedOptionsSet, which omit the oapp.key() they mutated.

RECOMMENDATION

Add a state: Pubkey field to each #[event] struct in the rbac crate and the oapp crate, and populate it from ctx.accounts.default_admin.key() (or the equivalent state field) at every emit_cpi! site.

RESOLUTION

LayerZero Team - Resolved.

INFO I-11 RESOLVED

The default_admin_role should not be dynamic.

CATEGORY Warning LOCATION contracts/common/utils/solana/rbac/src/traits.rs#L60-L62 REVIEW OApp · Main Review 1

DESCRIPTION

The rbac code intentionally forbids granting or revoking/renouncing the default admin role. But if RoleType::default_admin_role returns a different value because of override or upgrade, this invariant can be bypassed. For example, the new default_admin_role can be set as an already existing role. This will silently promote all the members of the previous role. The RoleMember for the previous default admin will become unreachable. This will also create the following asymmetry: begin_default_admin_transfer - remains callable only by the real default_admin

accept/grant/revoke/renounce - can be called by any of the holders of the new role

RECOMMENDATION

Consider documenting that default_admin_role must not change.

RESOLUTION

LayerZero Team - Resolved.

INFO I-12 RESOLVED

RoleRevoked.sender doc copy-pasted from grant.

CATEGORY Informational LOCATION contracts/common/utils/solana/rbac/src/events.rs#L19 REVIEW OApp · Main Review 1

DESCRIPTION

The sender field on the RoleRevoked event carries a doc comment that describes the field as if it belonged to RoleGranted. The comment refers to the operator who initiated a grant, even though the event is emitted on revocation.

RUST

```rust
#[event]
pub struct RoleGranted {
pub role: u8,
pub account: Pubkey,
/// The operator who initiated the grant (named `sender` for EVM parity).
pub sender: Pubkey,
}

#[event]
pub struct RoleRevoked {
pub role: u8,
pub account: Pubkey,
/// The operator who initiated the grant (named `sender` for EVM parity).
pub sender: Pubkey,
}
```

RECOMMENDATION

Consider updating the doc comment on RoleRevoked.sender to reflect the revoke semantic, for example

RUST

```rust
/// The operator who initiated the revoke (named `sender` for EVM parity).
pub sender: Pubkey,
```

RESOLUTION

LayerZero Team - Resolved.

INFO I-13 RESOLVED

Clarify endpoint program check in CPI wrappers.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs REVIEW OApp · Main Review 1

DESCRIPTION

Each CPI wrapper in endpoint_cpi.rs (register_oapp(), set_delegate(), send(), quote(), clear(), send_compose(), clear_compose()) carries a doc comment of the form

accounts[0] must be the Endpoint program (validated by construct_context).

The wording could be read to imply that construct_context verifies accounts[0] against the canonical LayerZero Endpoint program. In practice, the check inside construct_context (generated by cpi_helper 's CpiContext derive - see LayerZero-v2/packages/layerzero-v2/solana/programs/libs/cpi-helper/src/lib.rs#L51-L53) is:

RUST

```rust
if (program_id ≠ accounts[0].key()) {
return Err(InvalidProgramId.into());
}
```

program_id is bound to the endpoint_program: Pubkey parameter that the caller supplied. So the check confirms that the two caller-supplied values are consistent with each other, but it does not by itself establish that either one is the real Endpoint. The actual identity guarantee comes from wherever the outer OApp instruction sources endpoint_program - typically a hardcoded constant or a stored, RBAC-gated config field. This is not a vulnerability in the wrappers themselves; the design simply pushes the trust boundary one layer up to the caller. The current docstring may give readers (and future maintainers building OApps on top of these wrappers) a slightly stronger impression of where the guarantee lives than is accurate.

RECOMMENDATION

Consider rewording the docstrings so the trust boundary is explicit. For example

TEXT

```
/// # Accounts validation
/// - `accounts[0]` must equal the caller-supplied `endpoint_program` parameter.
```

RESOLUTION

LayerZero Team - Resolved.

INFO I-14 RESOLVED

Misleading Code Comment.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L16 REVIEW OApp · Main Review 1

DESCRIPTION

The comment in anchor-trait/src/lib.rs states, "See spec.md for the full specification." However, the repository does not include a spec.md file, making the comment misleading.

RECOMMENDATION

Update the comment to reflect the current repository structure, or add a spec.md file.

RESOLUTION

LayerZero Team - Resolved.

INFO I-15 RESOLVED

Endpoint CPI wrappers rely on account ordering.

CATEGORY Suggestion LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L24-L187 REVIEW OApp · Main Review 1

DESCRIPTION

The CPI wrappers in endpoint_cpi.rs (register_oapp(), set_delegate(), send(), quote(), clear(), send_compose(), clear_compose()) all rely on the caller passing the accounts slice in a specific positional order accounts[0] must be the endpoint program. accounts[1..N] must match the field declaration order of the corresponding cpi::accounts::* struct in endpoint-interface. This positional dependency comes from the cpi_helper::CpiContext derive macro at LayerZero-v2/packages/layerzero-v2/solana/programs/libs/cpi-helper/src/lib.rs#L22-L33, which generates a construct_context() implementation that pops accounts from the slice in field-declaration order and assigns each to the matching CPI accounts field by position. The wrappers in endpoint_cpi.rs further encode this dependency via hardcoded indices, e.g.: register_oapp() checks accounts[2] == oapp because oapp is the second field of RegisterOApp (after payer). set_delegate(), send(), clear(), send_compose(), and clear_compose() each check accounts[1] because their respective signer/identity field is declared first in the struct. If an OApp developer building on top of this framework is unaware of the strict positional contract, they could easily pass the accounts slice in a wrong order. The wrapper's own check would only catch the mismatch at the single hardcoded index it inspects (e.g., accounts[2] for register_oapp()); other slots could silently land in the wrong field. The endpoint's downstream account validation would eventually reject the malformed CPI, but the resulting error (a seed-derivation or ownership failure) is far less obvious than a layout mismatch and complicates debugging. The endpoint-interface git dependency in Cargo.toml is pinned by SHA, so the upstream struct order cannot drift unexpectedly. The remaining risk is purely a developer-awareness one: the wrappers' security guarantees depend on a layout contract that is implicit and undocumented at the call site.

RECOMMENDATION

Consider adding a brief comment on each wrapper (or at the top of the module) documenting the positional dependency on the corresponding cpi::accounts::* struct. For example

TEXT

```
/// NOTE: `accounts[2]` corresponds to the `oapp` field of `RegisterOApp` in
/// `endpoint-interface`. The full slice must follow the struct's field order:
/// [endpoint_program, payer, oapp, oapp_registry, system_program, ...remaining].
```

This makes the implicit assumption explicit at the call site, helping OApp developers lay out their accounts correctly the first time.

RESOLUTION

LayerZero Team - Resolved.

INFO I-16 ACKNOWLEDGED

OApp initialize() is frontrun-hijackable.

CATEGORY Frontrunning LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/mod.rs#L27

REVIEW OApp · Main Review 1

DESCRIPTION

The #[oapp] macro framework generates a set of admin-gated instructions (set_peer(), init_enforced_options(), set_enforced_options(), next_nonce(), etc.), but does not generate or scaffold an initialize() instruction. Each OApp implementer must hand-roll their own initialization handler that creates the OApp state PDA, calls endpoint::register_oapp() via CPI, and seeds the initial RBAC default-admin role.

Solana's runtime delays the visibility of newly-deployed programs by DELAY_VISIBILITY_SLOT_OFFSET = 1 slot, so every OApp deployment exposes an unavoidable window between solana program deploy confirmation and the deployer's own initialize() tx landing in a subsequent block. Atomic deploy-plus-initialize is not achievable through any client-side mechanism (including Jito bundles, which execute within a single slot and therefore cannot invoke a program deployed in that same slot). During the window, any signer can submit a competing initialize() and: 1. Become the OApp's DefaultAdmin (typically via #[rbac::init_default_admin(... admin = payer.key())], which the framework's RBAC layer publishes as the recommended pattern). 1. Set themselves (or any address) as the LayerZero endpoint delegate through the freely-chosen register_oapp parameter. 1. Permanently lock out the legitimate deployer - the state account's init constraint succeeds exactly once. After the race resolves in the attacker's favor, they control all admin-gated configuration: set_peer(), set_send_library(), set_receive_library(), init_enforced_options(), grant_role(), begin_default_admin_transfer(), etc.

The framework provides no built-in mechanism to mitigate this. There is no macro-generated initializer, no attribute that binds initialization to the program's upgrade authority, and no mention of the hazard in the framework documentation. Protocol teams integrating the #[oapp] macro must independently know about Solana's deploy-and-init race and harden their handler accordingly - a non-obvious requirement for teams whose primary background is EVM, where proxy constructors atomize the same operation.

RECOMMENDATION

Either (a) document the hazard in the OApp framework guide and demonstrate the upgrade-authority constraint pattern, or (b) provide a framework-level attribute (e.g. #[oapp::deployer_only_init]) that binds initialize() to the program's upgrade authority by adding a program_data: Account<'info, ProgramData> to the accounts struct with a program_data.upgrade_authority_address == Some(deployer.key()) constraint. The constraint is robust regardless of race-window size because every non-deployer initialize() reverts.

RESOLUTION

LayerZero Team - Acknowledged. The framework-level initialization risk remains; Console OFT handles it at the application level.

INFO I-17 RESOLVED

RBAC docs reference non-existent #[grant_role].

CATEGORY Documentation LOCATION contracts/common/utils/solana/rbac/src/lib.rs#L42 REVIEW OApp · Main Review 1

DESCRIPTION

The crate-level documentation in `lib.rs` instructs integrators to use macros that do not exist in the current API. Both the Quick Start prose and example show per-instruction attributes such as `#[grant_role(...)]`, `#[revoke_role]`, and `#[renounce_role]`, none of which are exported by `rbac_macros`.

RUST

```rust
//! ## Quick Start
//! 1. Define your role enum with `#[derive(RoleType)]`
//! 2. Define your `RoleMember` account
//! 3. Use `#[grant_role]`, `#[revoke_role]`, `#[renounce_role]`
//!
//! #[grant_role(program_state = Store, role_member = RoleMember, role_type = RoleType)]
//! pub struct GrantRole<'info> {}
```

The actual API uses the module-level `#[rbac(state = ..., role_type = ...)]` macro, accounts-struct attributes such as `#[only_role(...)]` and `#[init_default_admin(...)]`, the `RoleType` and `DefaultAdmin` derives, and the `#[rbac_instruction]` override marker. The grant, revoke, and renounce instructions are generated by `#[rbac]`, not by per-struct attributes.

RECOMMENDATION

Rewrite the Quick Start and Example sections in `lib.rs` to reflect the actual API.

RESOLUTION

LayerZero Team - Resolved.

INFO I-18 RESOLVED

deserialize_alt Can Panic Via unwrap.

CATEGORY Informational LOCATION apps/oapp-app/contracts/solana/src/common.rs#L135 REVIEW OApp · Main Review 1

DESCRIPTION

deserialize_alt contains a panic-style code path when reading ALT account data. In common.rs, it invokes alt.try_borrow_data().unwrap() prior to deserializing the address lookup table; as a result, a borrow failure causes an uncontrolled abort rather than returning a structured program error. As a result, transaction failure occurs via a panic rather than returning InvalidAddressLookupTable, leading to inconsistent error-handling behavior.

RECOMMENDATION

Consider replacing the unwrap with proper error propagation, and returning a controlled error on borrow failure.

RESOLUTION

LayerZero Team - Resolved.

INFO I-19 RESOLVED

Hardcoded crate path in declare_instructions.

CATEGORY Compatibility LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L98, #L121

REVIEW OApp · Main Review 1

DESCRIPTION

declare_instructions!() expands generated code with explicit::anchor_trait paths for both InstructionSet and InstructionSpec. This couples downstream users to one exact dependency name in Cargo.toml. If an integrating macro crate renames the dependency key, such as anchor-trait to my-anchor-trait, local imports can be updated successfully but the macro expansion still fails because::anchor_trait is no longer resolvable.

RECOMMENDATION

Replace hardcoded::anchor_trait references in declare_instructions!() with \$crate -qualified paths such as $crate::InstructionSet and $crate::InstructionSpec. This preserves correct resolution to the defining crate while remaining compatible with renamed dependencies in downstream crates.

RESOLUTION

LayerZero Team - Resolved.

INFO I-20 RESOLVED

Arbitrary marker accepted instead of (ctx).

CATEGORY Compatibility LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L60, #L85

REVIEW OApp · Main Review 1

DESCRIPTION

declare_instructions!() documents (ctx) as the syntax that selects the spec(&ctx) dispatch path, but the matcher accepts any identifier in that position. As a result, inputs such as (banana) are treated the same as (ctx) and silently route to the ctx-passing branch. This makes the macro interface looser than its documented contract and can hide typos or misunderstandings in downstream macro crates.

RECOMMENDATION

Restrict the ctx-passing branch to the literal (ctx) marker and reject any other identifier with an explicit compile-time error. This keeps the accepted syntax aligned with the documented API and prevents accidental acceptance of mistyped markers.

RESOLUTION

LayerZero Team - Resolved.

INFO I-21 RESOLVED

Instruction aliases emit duplicate entrypoints.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L188-L198

REVIEW OApp · Main Review 1

DESCRIPTION

The framework derives each generated entrypoint name from the instruction's canonical name. If two instruction variants are intentionally aliased to the same module in declare_instructions!(), they share the same canonical name and therefore generate the same public entrypoint function. This makes instruction aliases unsupported in practice, since code generation emits colliding entrypoints for both variants. In modules with default implementations, the same spec() and generator will also usually emit the same default accounts struct and handler impl twice. Even if an implementer forced different struct names through custom naming tricks, the generated entrypoints would still collide because the entrypoint name is tied to the shared canonical instruction name.

RECOMMENDATION

Clearly document that each instruction variant must use a unique module identity.

RESOLUTION

LayerZero Team - Resolved.

INFO I-22 RESOLVED

Docs understate default impl remains available.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/README.md#L36, #L207, #L220

REVIEW OApp · Main Review 1

DESCRIPTION

The documentation says the default accounts struct is always emitted and describes account extension as reusing the base implementation, but it does not state clearly enough that the original default implementation remains available after extension as well. An integrator can reasonably read the extension flow as a replacement path and assume the default implementation is no longer reachable once an extended accounts type is introduced. In practice, the framework emits both the original default struct and handler impl and the extended variant, so extension is additive rather than exclusive.

RECOMMENDATION

Update the documentation to state explicitly that extending default accounts does not replace or disable the original default implementation. The Provided and Extending Default Accounts sections should emphasize that both the original default struct and its handler impl remain emitted and usable unless the integrator deliberately routes all usage through the extended type.

RESOLUTION

LayerZero Team - Resolved.

INFO I-23 RESOLVED

Default accounts generics are not supported.

CATEGORY Compatibility LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L84, #L177

REVIEW OApp · Main Review 1

DESCRIPTION

Generic parameters on default accounts types are not supported by the framework's code generation paths. In the extend-accounts path, the generated macro reconstructs the type as pub struct $name<'info> and impl $name<'_>, which drops any additional generic parameters from the original default definition. In the default entrypoint path, the generated context and dispatch use only the bare accounts ident via Context<#accounts> and #accounts::#handler_fn, so a default accounts type such as StdShared<'info, T> cannot be instantiated correctly. As a result, default account definitions with extra generics may compile as standalone definitions but fail once the framework generates the corresponding entrypoint or extended variant.

RECOMMENDATION

Choose one explicit behavior and document it. If generic default account types are intended to be supported, preserve full generic type information in both the default entrypoint and extend-accounts generation paths rather than reducing the type to a bare ident or hardcoded <'info>. If they are not intended to be supported, validate and reject extra generics beyond the expected 'info form with a clear compile-time error and document that restriction.

RESOLUTION

LayerZero Team - Resolved.

INFO I-24 RESOLVED

Field-Injection Macros Always Append New Fields.

CATEGORY Documentation LOCATION contracts/common/utils/solana/rbac/README.md#L103 REVIEW OApp · Main Review 1

DESCRIPTION

The README states that #[only_role] appends role_member only if that field does not already exist, but the implementation does not perform that check. In contracts/common/utils/solana/rbac/README.md#L103, the generated field is described as being appended conditionally; however, the macro in only_role.rs unconditionally does fields.named.push(new_field). The same pattern exists in init_default_admin.rs for admin_role_member.

If a developer manually defines one of these fields and also applies the macro, the expansion produces duplicate field names, causing the struct to fail compilation. Although this does not pose a runtime risk, it is inconsistent with the README and may lead to confusing compile-time errors

RECOMMENDATION

Update the README to document this behavior.

RESOLUTION

LayerZero Team - Resolved.

INFO I-25 RESOLVED

OAppInfo can be silently uninitialized.

CATEGORY Compatibility LOCATION apps/oapp-app/contracts/solana/docs/oapp-info.md#L283 REVIEW OApp · Main Review 1

DESCRIPTION

OAppInfo is part of the shared OApp SDK integration contract, but its initialization is only an implicit expectation placed on application developers. The framework documents that developers should create the OAppInfo PDA and call init_oapp_info!() inside their own initialize instruction, but #[oapp] does not enforce this and does not generate a default initialization path. As a result, an OApp can be deployed in a functionally working on-chain state while still lacking the metadata required by the standardized SDK. If a developer is unaware of this requirement and later revokes upgrade authority, the deployment can become permanently incompatible with the shared SDK with no straightforward remediation path.

RECOMMENDATION

Consider adding an initialize macro which requires OAppInfo PDA to exist by default and provide an explicit opt-out for programs that intentionally do not want SDK compatibility. This approach protects devs from unknowingly creating their OApp without OAppInfo PDA, while still keeping maximum flexibility

RESOLUTION

LayerZero Team - Resolved.

INFO I-26 RESOLVED

Generated Code Is Susceptible To Name Shadowing.

CATEGORY Compatibility

LOCATION contracts/common/framework/solana/anchor-trait/src/generation.rs#L190, contracts/common/framework/solana/anchor-trait/src/generation.rs#L195

REVIEW OApp · Main Review 1

DESCRIPTION

The code generated by anchor-trait (including RBAC/OApp macros) relies on unqualified symbols such as Result, Context, and other Anchor types/macros resolved from the caller's scope. While RBAC documentation notes that anchor_lang::prelude::* is expected to be in scope, this does not prevent standard Rust name shadowing from affecting macro expansion. In particular, generated entrypoints use bare Result<T> rather than a fully qualified path (e.g., anchor_lang::Result<T>). Additionally, the assembled program module imports all parent-scope items via use super::*, allowing any user-defined aliases (e.g., type Result<T> = std::result::Result<T, ()>;) to propagate into the generated module. As a result, if a developer defines a conflicting alias such as Result, the generated code may resolve to the shadowed type instead of the intended Anchor type. This can lead to compile-time failures, even when user-authored handlers explicitly return anchor_lang::Result. The issue therefore stems from a macro hygiene limitation, where generated code depends on caller-scope name resolution rather than fully qualified paths.

RECOMMENDATION

Use fully qualified paths (e.g., anchor_lang::Result, anchor_lang::Context) in generated code to avoid reliance on caller-scope resolution and reduce the risk of name shadowing. Alternatively, explicitly document that shadowing core types (such as Result) in the parent scope is unsupported when using these macros.

RESOLUTION

LayerZero Team - Resolved.

LOW L-01 ACKNOWLEDGED

OAppInfo Can Reference Missing TLV Entry.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L246 REVIEW OApp · Main Review 2

DESCRIPTION

The TLV-based OAppInfo model relies on discriminator fields to describe which sibling TLV entry should exist next, but that relationship is not validated during initialization. In particular, init_oapp_base! writes OAppBase.app_discriminator, while the actual sibling entry is written separately through init_tlv_entry. Nothing in this flow requires the referenced TLV entry to be present before initialization succeeds. As a result, a malformed or incomplete initializer can leave OAppInfo in an internally inconsistent state where the base entry advertises an app-specific discriminator, but no matching sibling TLV entry exists in the account. This does not appear to create a direct authorization or fund-loss issue, but it can break SDK discovery, off-chain parsing, and transaction construction for application-specific flows that rely on the discriminator chain being valid.

RECOMMENDATION

Add a validation or higher-level initialization helper that ensures any non-zero app_discriminator in OAppBase corresponds to a sibling TLV entry before initialization succeeds.

RESOLUTION

LayerZero Team - Acknowledged. The generic primitive was kept unchanged, with applications responsible for initializing the full TLV chain.

INFO I-01 RESOLVED

Unconstrained sender in setup_default_admin!.

CATEGORY Events LOCATION contracts/common/utils/solana/rbac/macros/src/setup_default_admin.rs#L102

REVIEW OApp · Main Review 2

DESCRIPTION

setup_default_admin!(ctx, state, admin, role_type, sender) parses sender as an arbitrary Expr and forwards it unmodified into the RoleGranted event

RUST

```rust
emit_cpi!(::rbac::events::RoleGranted {
state: #ctx.accounts.#state.key(),
role: <#role_type as ::rbac::traits::RoleType>::default_admin_role().into(),
account: admin_pubkey,
sender: #sender,            // arbitrary developer expression
});
```

Nothing constrains sender to a Signer -typed field, the payer, or any account related to the grant. A caller can pass Pubkey::default(), system_program.key(), or any pubkey, and the resulting event is well-formed and indexable. This is the only RoleGranted emission site where sender is not hard-coded to ctx.accounts.authority.key(), breaking the convention used by grant_role and accept_default_admin_transfer.

RECOMMENDATION

Consider enforcing that the passed sender is a signer.

RESOLUTION

LayerZero Team - Resolved.

INFO I-02 RESOLVED

clear wrapper docs imply OApp-only signer.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L283 REVIEW OApp · Main Review 2

DESCRIPTION

The Endpoint program's Clear accounts struct authorizes the signer account through a disjunction

RUST

```
constraint = signer.key() == params.receiver
|| signer.key() == oapp_registry.delegate @LayerZeroError::Unauthorized
```

The endpoint deliberately allows either the OApp PDA itself OR the OApp's registered delegate to clear an inbound payload. The framework's CPI wrapper, however, presents an API that strongly implies only the OApp PDA can be the signer:

RUST

```
/// # Accounts validation
/// - `accounts[0]` must equal the caller-supplied `endpoint_program` parameter.
/// - `accounts[1]` must match the provided `receiver` key (the OApp PDA).
pub fn clear(
endpoint_program: Pubkey,
receiver: Pubkey,
accounts: &[AccountInfo],
seeds: &[&[u8]],
params: ClearParams,
) → Result<[u8; 32]> {
validate(accounts, endpoint_program, CLEAR_MIN_ACCOUNTS_LEN)?;
require_keys_eq!(accounts[1].key(), receiver, ErrorCode::ConstraintAddress);
...
}
```

The parameter is named receiver and the docstring labels it "(the OApp PDA)". A delegate-clears flow IS still reachable through this wrapper because the caller controls what they pass as the receiver argument (the wrapper only checks that accounts[1] matches whatever the caller claims, not that the value actually equals the OApp address), but a developer reading the wrapper signature and docstring has no way to know that.

RECOMMENDATION

Consider updating the wrapper's docstring to make the underlying endpoint behavior explicit, note that the endpoint accepts either the OApp PDA or the registered delegate as the signer, that accounts[1] must be whichever signer the caller intends to use, and that params.receiver must always be the OApp address regardless of who signs.

RESOLUTION

LayerZero Team - Resolved.

INFO I-03 RESOLVED

Stale OAppInfo Documentation After TLV Migration.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/state_types.rs#L6, apps/oapp-app/contracts/solana/docs/oapp-macro.md#L105

REVIEW OApp · Main Review 2

DESCRIPTION

The repository's OAppInfo documentation is inconsistent with the current implementation. While the implementation has moved to a raw SPL TLV-backed OAppInfo account with OAppBase as the framework entry and app-specific metadata appended as sibling TLV entries, older documentation still describes OAppInfo as a monolithic versioned PDA containing InstructionVersions plus an extra_info: Vec<u8> extension blob. This older model is still described in apps/oapp-app/contracts/solana/README.md#L96-L97. The #[oapp] macro documentation is stale in the same area. apps/oapp-app/contracts/solana/docs/oapp-macro.md#L105 still states that the macro prelude auto-generates an OAppInfo account struct, but the current macro emits only OAppPeer and EnforcedOptions in apps/oapp-app/contracts/solana/macros/src/oapp.rs#L67-L69. In the current design, oapp_info must be declared manually as UncheckedAccount<'info>, sized explicitly for TLV entries, and initialized with oapp::init_oapp_base! plus any app-specific init_tlv_entry(...) calls. Some internal and example comments still reinforce the old model as well.

RECOMMENDATION

Update all stale OAppInfo references to consistently describe the current manual UncheckedAccount<'info> plus TLV-entry model, and remove outdated references to auto-generated OAppInfo, extra_info, init_oapp_info!, and OAppInfoEntry

RESOLUTION

LayerZero Team - Resolved.

INFO I-04 RESOLVED

anchor-trait README stale on wrapper_defs.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/README.md#L206 REVIEW OApp · Main Review 2

DESCRIPTION

The assembly.rs change in the latest sync moved wrapper_defs inside the generated module and updated the inline ASCII diagram in the source doc-comment to reflect the new layout. The "Assembly output structure" diagram in anchor-trait/README.md was not updated and still shows wrapper_defs as a top-level emission alongside prelude and default_impls, which contradicts the current code.

The README still says:

RUST

```rust
<prelude>
<wrapper_defs>       ← shown outside the module
<default_impls>
...
[vis] mod <name> { ... }
```

The actual layout, per assembly.rs:

RUST

```rust
prelude
default_impls
...
mod name {
filtered_items
wrapper_defs              ← now emitted inside, so handlers can reach
//   the user's nested same-module helpers
entrypoints
}
```

RECOMMENDATION

Mirror the updated diagram from the assembly.rs doc-comment into the README so the two stay aligned.

RESOLUTION

LayerZero Team - Resolved.

INFO I-05 RESOLVED

Consider Size Check In update_tlv_entry.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L189-L200 REVIEW OApp · Main Review 2

DESCRIPTION

update_tlv_entry is only safe when the new value has the same serialized length as the existing TLV entry. The code comments document this and recommend using it for fixed-size entries, but the function itself does not enforce that invariant. Testing confirmed that misuse has non-obvious failure modes. If a shorter variable-length value is written, the update can succeed while leaving stale trailing bytes in the slot and preserving the original TLV length. If a longer value is written, the update fails, but only after partially overwriting the existing bytes, which can leave the TLV entry corrupted and no longer decodable. This does not appear to impact the current in-scope metadata structs, which are effectively fixed-size, but it remains an integration footgun for downstream developers.

RECOMMENDATION

Consider rejecting updates when the new serialized length differs from the existing allocated length.

RESOLUTION

LayerZero Team - Resolved.

INFO I-06 RESOLVED

Incorrect composer docs link.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs#L15

REVIEW OApp · Main Review 2

DESCRIPTION

The composer macro generator for lz_compose_types_info() links to the Solana OApp receive-types documentation at https://docs.layerzero.network/v2/developers/solana/oapp/overview#how-lz_receive_types_v2-works. This file documents composer-specific discovery using LzComposeParams, LzComposeTypesV2Accounts, and LZ_COMPOSE_TYPES_SEED, so the current link sends implementers to the unrelated lz_receive_types_v2() flow. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Replace the See link with the composer-specific documentation URL: https://docs.layerzero.network/v2/developers/solana/composer/overview#lz_compose_types_info.

RESOLUTION

LayerZero Team - Resolved.

INFO I-07 RESOLVED

Private repo link in source comment.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/lib.rs#L32 REVIEW OApp · Main Review 2

DESCRIPTION

The comment immediately before declare_program!() links to https://github.com/LayerZero-Labs/monorepo-internal/..., which appears to point to a private repository or PR discussion. This does not affect protocol behavior, but external reviewers may be unable to inspect the linked context for why idls/endpoint.json is mirrored in this package.

RECOMMENDATION

Be aware that the source includes a reference to private development context. If this source is intended for broader public consumption, consider replacing the private URL with a short public note explaining why idls/endpoint.json is mirrored before declare_program!().

RESOLUTION

LayerZero Team - Resolved.

INFO I-08 RESOLVED

Incomplete package files list.

CATEGORY Configuration LOCATION apps/oapp-app/contracts/solana/package.json#L7-L14 REVIEW OApp · Main Review 2

DESCRIPTION

The npm package whitelist in package.json includes the entire macros directory while omitting idls. Including macros publishes the macros/examples tree even though consumers only need macros/src and macros/Cargo.toml. The examples are also large, which can materially increase package size. More importantly, the package now uses declare_program!(endpoint), which reads idls/endpoint.json at compile time. If the package is installed from the published artifact without idls, consumers may fail to compile with a missing idls directory or accidentally resolve an unrelated ancestor idls/endpoint.json.

RECOMMENDATION

Keep the package whitelist limited to compile-time inputs and runtime source

JSON

```
"files": [
"src",
"idls",
"macros/src",
"macros/Cargo.toml",
"Cargo.toml",
"Anchor.toml",
"rust-toolchain.toml",
"rustfmt.toml"
]
```

Do not include macros wholesale or build artifacts such as macros/target; include macros/examples only if examples are intentionally shipped.

RESOLUTION

LayerZero Team - Resolved.

INFO I-09 RESOLVED

register_oapp Prerequisite Documentation Gap.

CATEGORY Documentation

LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/oapp/accept_default_admin_transfer.rs#L16-L19

REVIEW OApp · Main Review 2

DESCRIPTION

The #[oapp] macro unconditionally injects an accept_default_admin_transfer override that calls set_delegate, which requires the OAppRegistry PDA to have been created via register_oapp. This prerequisite is documented in the macro implementation comment in accept_default_admin_transfer.rs

TEXT

```
//! **`register_oapp` is a mandatory call in the OApp's `initialize` handler.**
//! Omitting it means `accept_default_admin_transfer` will always fail once a
//! default-admin transfer is initiated, permanently bricking the admin transfer
//! flow.
```

However, this prerequisite is not surfaced alongside the override description in the main user-facing docs in README and oapp-macro.md. The bundled counter example also omits register_oapp in counter/src/lib.rs, which can reinforce the gap for integrators reading the example first.

RECOMMENDATION

Document the register_oapp requirement in the main #[oapp] docs and examples wherever the generated accept_default_admin_transfer override is described.

RESOLUTION

LayerZero Team - Resolved.

INFO I-10 RESOLVED

OAppInfo init accepts nonzero TLV tail.

CATEGORY Validation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs#L183-L185, #L254 REVIEW OApp · Main Review 2

DESCRIPTION

init_oapp_base!() initializes the framework OAppBase TLV entry through init_tlv_entry(), which directly calls TlvStateMut::unpack() and then alloc_and_pack_variable_len_entry(). The SPL TLV parser treats the first uninitialized discriminator as the end of initialized TLV data, so nonzero bytes after that point are not rejected before the new entry is written. If init_oapp_base!() is called on an OAppInfo account whose data is not freshly zero-filled, only the byte range occupied by the new OAppBase record is overwritten. Any remaining nonzero tail bytes are preserved. Those bytes may later cause parsing failures or, if they happen to form valid TLV records, be interpreted as unintended sibling entries by SDKs or other readers. This is not expected in the normal init flow because newly allocated Solana account data is zero-filled, but the framework helper itself does not document or validate this assumption.

RECOMMENDATION

Either acknowledge and document that init_oapp_base!() must only be called on a freshly initialized, zero-filled OAppInfo account, or consider adding additional validation before OAppBase initialization to reject nonzero bytes after the first uninitialized TLV slot.

RESOLUTION

LayerZero Team - Resolved.

INFO I-11 RESOLVED

Program attr filtering should be documented.

CATEGORY Documentation LOCATION contracts/common/framework/solana/anchor-trait/src/assembly.rs#L81-L82

REVIEW OApp · Main Review 2

DESCRIPTION

Assembly::assemble() removes any module attribute whose final path segment is program before partitioning attributes into inner and outer attributes. This supports re-emitting a single managed Anchor #[program] attribute, including deduplicating qualified forms such as #[anchor_lang::prelude::program]. This behavior can also remove other attributes whose path ends in program, even if they are not Anchor's #[program] attribute. Because filtering happens before the AttrStyle::Inner partition, it also applies to inner attributes. Anchor's #[program] attribute is an outer procedural macro on the program module, while #![program] as an inner macro attribute is not a stable/usable Anchor program form. This is not necessarily incorrect if the framework intentionally reserves any attribute path ending in program, but that convention is not currently explicit to consumers.

RECOMMENDATION

Document that anchor-trait treats outer module attributes ending in program as reserved for the managed Anchor #[program] emission and may remove them during assembly. Also consider applying the program filtering only after partitioning attributes, so inner attributes are preserved while outer Anchor #[program] attributes are still deduplicated.

RESOLUTION

LayerZero Team - Resolved.

INFO I-12 RESOLVED

RoleType derive can use nonzero default.

CATEGORY Validation LOCATION contracts/common/utils/solana/rbac/macros/src/role_type.rs#L30-L46

REVIEW OApp · Main Review 2

DESCRIPTION

role_type_derive_impl() only emits the discriminant-zero assertion when it finds a #[default] variant. If no #[default] variant exists, default_assert is empty and the macro still emits impl::rbac::traits::RoleType for #name {}. Since RoleType requires Default and default_admin_role() returns Self::default(), a consumer can manually implement Default so that #[derive(RoleType)] uses a nonzero default-admin discriminant. In that case, seed() derives the default-admin role byte from the nonzero discriminant, while the docs and SDK compatibility guidance expect the default-admin PDA seed to be [0].

RECOMMENDATION

Either be aware of and document that #[derive(RoleType)] only enforces the zero-discriminant invariant when a # [default] variant is present, or change role_type_derive_impl() to require a #[default] variant and reject derives where it is missing. contracts/common/utils/solana/rbac/src/traits.rs#L65-L67explains how devs may still want to use different default discriminant than 0, but it says implement this method yourself which points to default_admin_role. Adjust the comment If the intention is to let the dev have different discriminant than 0 by using the impl Default for... syntax.

RESOLUTION

LayerZero Team - Resolved.

INFO I-01 ACKNOWLEDGED

Composer codegen doc points to the wrong file.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs#L16

REVIEW OApp · Remediation Review 1

DESCRIPTION

The doc comment on the composer macro generator at apps/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose_types_info.rs points to the wrong documentation file

RUST

```rust
//! This follows the same V2 account-discovery model as `lz_receive_types_info`.
//! See `docs/lz-receive-types.md#lzcomposetypes-v2-differences` for the
//! compose-specific differences.
```

The comment promises "the compose-specific differences" but links to docs/lz-receive-types.md, which contains no such section. Those differences live in docs/lz-compose-types.md under the heading ### Differences from LzReceiveTypes. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Repoint the reference to the correct file

RUST

```rust
//! See `docs/lz-compose-types.md` for the compose-specific differences.
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO I-02 ACKNOWLEDGED

Typo in CallerNotPendingAdmin doc comment.

CATEGORY Documentation LOCATION contracts/common/utils/solana/rbac/src/errors.rs#L7 REVIEW OApp · Remediation Review 1

DESCRIPTION

The doc comment on the CallerNotPendingAdmin error variant at contracts/common/utils/solana/rbac/src/errors.rs contains a typo and incorrect terminology

RUST

```rust
    /// The caller to accept the default admin transfer neds to be the pending default owner
    #[msg("Caller is not the pending admin")]
    CallerNotPendingAdmin,
```

neds should be needs. Additionally, the comment refers to the actor as the pending default "owner", but the RBAC module models access control in terms of admins, not owners, the variant is CallerNotPendingAdmin, its message is "Caller is not the pending admin", and the staged value lives in pending_default_admin. This is a documentation-only issue and does not affect generated code or runtime behavior.

RECOMMENDATION

Fix the typo and align the wording with the rest of the module

RUST

```rust
    /// The caller of accept_default_admin_transfer must be the pending default admin.
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO I-03 ACKNOWLEDGED

Broken intra-doc link to CodegenContext.

CATEGORY Best Practices LOCATION contracts/common/framework/solana/anchor-trait/src/lib.rs#L63

REVIEW OApp · Remediation Review 1

DESCRIPTION

The doc comment on the declare_instructions! macro at contracts/common/framework/solana/anchor-trait/src/lib.rs:63 uses rustdoc intra-doc-link syntax (square brackets) for CodegenContext

RUST

```rust
/// Appending `(ctx)` after `mod_name` routes to `mod_name::spec(&ctx)`
/// with the [`CodegenContext`] reference; omitting it calls `spec()` with
```

The [...] form instructs rustdoc to resolve CodegenContext to a real item and link to it. No such item exists inside the anchor-trait crate, CodegenContext is only a placeholder name that downstream callers choose for their own context struct (the macro's example passes it via context = CodegenContext;, and the associated type is referenced elsewhere as the inert code-span CodegenContext at line 42). As a result cargo doc emits a rustdoc::broken_intra_doc_links diagnostic:

TEXT

```
warning: unresolved link to `CodegenContext`
--> src/lib.rs:63:16
|
63 | /// with the [`CodegenContext`] reference; omitting it calls `spec()` with
|                ^^^^^^^^^^^^^^^ no item named `CodegenContext` in scope
```

By default this is a warning, but it becomes a hard error: could not document anchor-trait (non-zero exit) under any deny escalation.

RECOMMENDATION

Drop the brackets so the name is inert text, matching the existing inert usage at line 42

RUST

```rust
/// with the `CodegenContext` reference; omitting it calls `spec()` with
```

RESOLUTION

LayerZero Team - Acknowledged.

INFO I-04 ACKNOWLEDGED

Misleading clear documentation.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/endpoint_cpi.rs#L271-L274

REVIEW OApp · Remediation Review 1

DESCRIPTION

The following documentation was added to endpoint_cpi::clear() in response to I-02

TEXT

```
/// - `accounts[1]` is the `signer` - pinned to `receiver` (the OApp PDA), which must also sign via
/// `seeds`. The Endpoint itself also accepts the registered delegate as `signer`; that path is
/// intentionally not exposed through this wrapper. To clear as the delegate, CPI to
/// `endpoint::cpi::clear` directly.
```

It says the wrapper doesn't expose the path that allows using delegate as a signer, but that is not true. There are actually 2 receivers. The first is the parameter the clear is accepting - the one that's validated to be equal to accounts[1] and is passed as signer. And the second is params.receiver. This second receiver will be the OApp PDA, while the first one can be either the PDA or the delegate. If the user passes the delegate as receiver, the endpoint would accept it.

RECOMMENDATION

Either change the documentation to make it clear both paths are accessible or compare receiver against params.receiver if the intention is to truly disable the delegate path.

RESOLUTION

LayerZero Team - Acknowledged.

INFO I-01 ACKNOWLEDGED

Stale OAppBase doc snippet in oapp-info.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/src/oapp_info.rs REVIEW OApp · Remediation Review 2

DESCRIPTION

The embedded OAppBase / IdlVersion code snippet in docs/oapp-info.md does not match the actual struct in src/oapp_info.rs. The doc was not synced when the struct was written, so a reader copying or trusting the snippet sees the wrong definition. Two mismatches: 1. The schema_version doc comment carries an extra sentence the real code does not have. Doc (docs/oapp-info.md):

```rust
    /// Borsh schema version of this entry. Bumps when fields are added,
    /// removed, reordered, or change type. The SDK reads this first to
    /// select the correct parser for the remaining bytes.
    pub schema_version: u8,
```

Real code (src/oapp_info.rs):

```rust
    /// Borsh schema version of this entry. The SDK reads this first to select the correct parser
    /// for the remaining bytes.
    pub schema_version: u8,
```

1. The IdlVersion derive list is missing Debug. Doc (docs/oapp-info.md):

```rust
    #[derive(Clone, Copy, AnchorSerialize, AnchorDeserialize, InitSpace, PartialEq, Eq)]
    pub struct IdlVersion {
    pub major: u8,
    pub minor: u8,
    }
```

Real code (src/oapp_info.rs):

```rust
    #[derive(Clone, Copy, AnchorSerialize, AnchorDeserialize, InitSpace, PartialEq, Eq, Debug)]
    pub struct IdlVersion {
    pub major: u8,
    pub minor: u8,
    }
```

RECOMMENDATION

For #1, consider updating the code to reflect the docs. For #2, update the docs and add the missing Debug.

RESOLUTION

LayerZero Team - Acknowledged.

INFO I-02 RESOLVED

OAppInfo v1 Can Allow Misleading Discovery.

CATEGORY Documentation LOCATION apps/oapp-app/contracts/solana/docs/oapp-info.md REVIEW OApp · Remediation Review 2

DESCRIPTION

OAppInfo v1 defines a fixed-template discovery model for provided standard OApp/RBAC instructions. SDKs derive required accounts from the published idl_version and predefined instruction templates, and schema v1 does not provide a per-instruction override mechanism for those templates. At the same time, the macro/framework layer continues to support extending or fully overriding provided standard instructions with custom account contexts. This creates a mismatch at the integration boundary: a program can publish OAppInfo v1 while changing the actual account layout of an instruction such as set_peer in a way that the v1 metadata format cannot express. A generic SDK or off-chain integrator relying on the advertised standard template may then construct an invalid transaction even though the program behaves as implemented. The current documentation describes the individual pieces but does not make this incompatibility explicit enough. oapp-info.md states v1 has no override path for standard instruction templates, while oapp-macro.md and anchor-trait documentation separately describe extending or overriding provided instructions. There is no clear statement that programs publishing OAppInfo v1 for standardized discovery must not extend or fully override the account contexts of SDK-discoverable provided standard instructions.

RECOMMENDATION

Explicitly document that programs publishing OAppInfo v1 must not extend or fully override the accounts of standard provided instructions.

RESOLUTION

LayerZero Team - Resolved.

LOW **L-02** RESOLVED

The compose docs say lz_compose has no sender to check, but anyone can queue a compose message

CATEGORY	Documentation
LOCATION	apps/project-types/oapp-app/contracts/solana/docs/lz-compose-types.md:21-22, apps/project-types/oapp-app/contracts/solana/macros/src/instructions/composer/lz_compose.rs, apps/project-types/oapp-app/contracts/solana/macros/src/lib.rs
REVIEW	Main Review

DESCRIPTION

The Endpoint's `send_compose` can be called by any signer, and the caller becomes the message's `from`, so anyone can queue a compose message to any composer. In `lz_compose`, the composer's `clear_compose` CPI only proves that the message really came from `params.from`. Whether that sender is a trusted OApp is the composer's job, just as `lz_receive` must match `params.sender` to the stored peer. This release's `lz-compose-types.md` says the opposite: "there is no peer to check, since `clear_compose` itself binds the message to its sender". The receive macro documents its check under `# Security` (`lz_receive.rs:6-17`, `lib.rs:33-45`), but the composer macro has no such note, and the example composer checks only `params.to`:

```rust
#[account(
mut,
seeds = [COMPOSER_SEED],
bump = composer_store.bump,
constraint = composer_store.key() == params.to,
)]
pub composer_store: Account<'info, ComposerStore>,
```

A composer built from these docs and examples acts on any compose message addressed to it. An attacker calls `send_compose` with their own wallet as `from` and a crafted payload, then calls the composer's `lz_compose`, and `clear_compose` succeeds. The same rule is stated elsewhere in this codebase and by LayerZero: the [EVM composer docs](https://docs.layerzero.network/v2/developers/evm/composer/overview) require `require(_oApp == oApp, ...)` alongside the Endpoint caller check, `MultiHopComposer.sol:80-81` enforces both, and the Stellar Endpoint's `send_compose` says "The composer MUST assert the sender because anyone can send compose msg with this function" (`messaging_composer.rs:23`). No in-scope program

L-02 The compose docs say lz_compose has no sender to check, but anyone can queue a compose message CONTINUED

is a composer, so the risk falls on third-party composers built on this framework. V1 said nothing about the sender; the "no peer to check" line is new.

RECOMMENDATION

Replace the "no peer to check" sentence with a statement that `clear_compose` only authenticates `params.from`, and that `lz_compose` must require `params.from` to be a trusted OApp such as a stored OFT key. Add a matching `# Security` section to the composer macro and `#[composer_instruction]`, and show the check in the example composers.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-04 RESOLVED

The README says a permanently blocked receive can be skipped, but skip cannot reach a verified payload

CATEGORY	Documentation
LOCATION	apps/project-types/console-oft-app/contracts/solana/README.md:150-157, apps/project-types/oapp-app/contracts/solana/README.md:72, apps/project-types/oapp-app/contracts/solana/src/endpoint_cpi.rs:burn()
REVIEW	Main Review

DESCRIPTION

This release refuses receives while a Token-2022 transfer fee is live, and the README tells operators that a live fee with a renounced fee authority "blocks them for good, leaving only `skip` or `nilify`". A receive that fails in `lz_receive` is already verified, and the Endpoint's `verify` advances the inbound nonce past every contiguously verified payload (`verify.rs:76`, `messaging_channel.rs:47-64`). The Endpoint's `skip` only accepts the next unverified nonce:

```rust
#[account(
mut,
seeds = [
NONCE_SEED,
params.receiver.as_ref(),
&params.src_eid.to_be_bytes(),
&params.sender[..]
],
bump = nonce.bump,
constraint = params.nonce == nonce.inbound_nonce + 1 @LayerZeroError::InvalidNonce
)]
pub nonce: Account<'info, Nonce>,
```

An operator who follows the README calls `skip` on the stuck payload and gets `InvalidNonce`. `nilify` works but only parks the payload: it sets the hash to nil, so the payload stays undeliverable and can be verified again (`nilify.rs:56-66`). The instructions that dispose of it are `burn` and `clear`, both available through the `MessagingChannelManager` wrappers. Both require `nonce <= inbound_nonce` (`burn.rs:25`, `clear.rs:29`), which every fee-blocked payload already meets, because `lz_receive` clears it on the Endpoint before

I-04 The README says a permanently blocked receive can be skipped, but skip cannot reach a verified payload CONTINUED

crediting. `skip` is for a different case: a verified payload above a nonce gap, where skipping the missing nonce lets the inbound nonce advance past it. The new OApp README row and the `endpoint_cpi::burn` doc describe `burn` without its nonce condition.

RECOMMENDATION

In README:156, replace "leaving only `skip` or `nilify`" with guidance to dispose of the stuck payloads with `burn` or `clear`, or to park them with `nilify`, and state that `skip` only applies to the next unverified nonce, for example to close a gap below a verified payload.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an “endorsement” or “disapproval” of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any “product” or “asset” created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

About Guardian.

Founded in 2022 by DeFi experts, Guardian is a leading audit firm in the DeFi smart contract space. With every audit report, Guardian upholds best-in-class security while achieving our mission to relentlessly secure DeFi.

WEB
guardianaudits.com

AUDITS
github.com/GuardianAudits/Audits

CONTACT
audits@guardianaudits.com